Abstract

The Patient-Doctor Management System (PDMS) is an offline, database-driven, desktop application designed specifically for small rural clinics in the Philippines to digitize everyday clinic tasks. These are the clinics where internet connectivity, budget, and infrastructure are evidently major constraints. The system is built using Python Tkinter and Microsoft SQL Server, which implements two role-isoled portals, Patient and Doctor. It enforces Role-Based Access Control across modules: appointment scheduling, medical record management, prescription generation, and billing. The database is normalized up to Third Normal Form, with data validations and secure transactions to safeguard data integrity. Several tests showed that all objectives were met, which makes the PDMS a working and extensible prototype for future upgrades.

Table of Contents

Title Page 1
Acknowledgement 2
Abstract 3
Table of Contents 4
Chapter 1 5
1.1 Background and Justification 5
1.2 Aims and Objectives 6
1.3 Structure of the Report 6
Chapter 2 8
2.1 Conceptual Review 8
2.1.1. Healthcare Information System and Electronic Medical Records 8
2.1.2 Role-Based Access Control 8
2.1.3 Appointment and Billing Systems 8
2.2 Theoretical Framework 9
2.3 Empirical Review 10
2.3.1 EMR Systems in the Philippines and Other Countries 10
2.3.2 Offline versus Cloud-Based Systems in Rural Healthcare 10
2.4 Research Gaps 10
2.5 Evaluation of Methods 10
Chapter 3 11
3.1 Requirements Analysis 11
3.2 MoSCoW Prioritization 12
3.3 Use Case Diagram 13
3.4 Activity Diagrams 14
3.5 Sequence Diagrams 17
Chapter 4 19
4.1 System and Architectural Design 19
4.2 Database Design 21
4.3 GUI and Algorithm Design 23
Chapter 5 29
5.1 Database Population 29
5.2 Code and Interface Implementation 31
5.3 Challenges and Resolutions 41
Chapter 6 43
6.1 Black-Box Testing 43
6.2 Boundary Testing 44
6.3 Performance Testing 44
6.4 Heuristic Evaluation 45
6.5 Predictive Assessment (ISSM/TAM) 45
Chapter 7 47
7.1 Analysis of Objectives 47
7.2 System Strengths and Limitations 47
7.3 Recommendations 48
References 49
Appendices 52
Word Count Breakdown 52
Appendix A: Use Case Descriptions 52
Appendix B: Data Dictionary 58
Appendix C: Data Validation and Exception Handling Evidence 60
Appendix D: AI Usage Documentation 63
Appendix E: Development Timeline with Research Advisor 66
Statement and Confirmation of Own Work 67

Chapter 1

Healthcare is an industry where administrative inefficiency is not merely an inconvenience; it can threaten patient lives. At its core, accurate and secure patient data is survival. The transition from paper-based records to electronic medical records (EMRs) has been a remarkable revolution in healthcare documentation, due to its increased effectiveness and accessibility, especially in hospitals. However, this adoption has been a challenge for small and rural clinics in the Philippines, especially in areas with fewer healthcare resources. The Department of Health (DOH) reports that 40% of Filipinos reside in rural areas, yet only 10% of healthcare providers serve in these communities (Leyso and Umezaki, 2024). But among this small percentage, a significant question arises: are they all equipped with a reliable health information management system?

1.1 Background and Justification

Macabasag et al. (2022) state that the Philippines functions under an “ambivalent health information management (HIM) regime,” reported by the Rural Health Units (RHUs). This is because healthcare EMR systems do not align with the realities of small clinics. Factors such as budget and infrastructure constraints, lack of training, and resistance to change hinder the successful deployment of an EMR system (Tinam-isan & Naga, 2024; Acacio-Claro, 2024). Most of the commercial systems, such as JuanHealth EMR and MD Clinic, have high subscription fees and maintenance (DoktorEMR, 2025). Moreover, these platforms are commonly built with cloud-based systems (Microsoft EHRs), requiring high-bandwidth internet connectivity and highly trained staff. These conditions are infeasible given the rural capacities.

While there are existing open-source local EMR (CHITS), the nationwide healthcare recording system (FHSIS) is still paper-based. According to Ongkeko et al. (2016), retrieving a patient record can take 30 minutes, and creating monthly reports can consume up to 10 hours in Quezon and geographically isolated and disadvantaged areas (GIDAs). This is due to the unmet infrastructure prerequisites, wherein the EMR system latency increases, resulting in “data-entry fatigue” and a decline in user acceptance (Ebardo and Celis, 2019; De Mesa et al., 2024). Consequently, clinics return to a paper-based workflow or an insecure spreadsheet, which is susceptible to manual errors and fragmented patient records.

Based on the synthesis, the researcher is motivated to develop a balanced and effective system: The Patient-Doctor Management System (PDMS) — built in Python with a Tkinter interface at PyCharm Community Edition 2024.3 IDE, designed to be intuitive and lightweight. For secure data storage and retrieval, it integrates the SQL Server through SQL Server Management Studio (SSMS) 20 environment. The system does not depend on network stability or heavy, cloud-based platforms. Python and MySQL stack is a reasonable choice, since it addresses the infrastructure gap. Marbella’s (2024) and Manikpuri’s (2025), and similar studies are PHP HMS models that depend on extensive setup and API services, which are expensive and not viable for local clinics.

1.2 Aims and Objectives

In response to the prevalent issue reported by RHU, the researcher aims to design and develop a localized, user-friendly, and secure clinic management system to overcome bottlenecks in patient data management workflow. experienced by small and rural clinics.

Objectives	Measurement Criteria
1. Develop a database-driven and fully offline clinic management prototype. 2. Retrieve patient data in under 3 seconds. 3. Implement a Role-Based Access Control with isolated portals.	Iterative Design and Development – Week 13-15
4. Design an intuitive GUI that enables consultations under 5 minutes with fewer than 12 clicks to process new information. 5. Accomplish a 100% data entry accuracy and a view response time of under 2.5 seconds. 6. Reduce appointment booking time from 10 minutes to 5 minutes.	Integration and Usability Testing – Week 22
7. Apply techniques in system development, SDLC, and database normalization. 8. Gain hands-on experience in healthcare IT for career opportunities.	Functional System, Documentation, and Final Report – Week 24

1.3 Structure of the Report

Chapter 1 discusses the prevalent challenges in small and rural clinics in the Philippines. It justifies the need for a localized and offline system. Qualitative and quantitative objectives are stated.

Chapter 2 reviews HIS and EMR systems with their relevant modules and the frameworks that support the study. Empirical evidence compared local and global HIS adoption and offline versus cloud-based systems. Research gaps and methods assist in the development.

Chapter 3 establishes functional and non-functional requirements. It also presented the MoSCoW prioritization and UML diagrams created in the first few months of the project.

Chapter 4 documents the system design, such as the three-tier architecture, role-based GUI wireframes, normalized database schema, and algorithm designs of each module before the implementation.

Chapter 5 narrates the five-month iterative development of PDMS, including the database setup, Tkinter windows, data validations, and resolution of technical challenges encountered.

Chapter 6 presents the tests conducted to validate the system, including black-box testing, objective-based performance testing, boundary testing, and heuristic evaluation, which are then analyzed against the frameworks.

Chapter 7 provides reflection and analysis on the eight objectives that were achieved; limitations and recommendations, like online payment integration and formal pilot testing, were identified.

Ultimately, dependence on paper-based records in rural and small Philippine clinics can adversely affect patient safety due to human error and unorganized management. While mainstream and cloud-based EMRs are powerful, they are often not feasible due to clinics' limitations. Such claims will be further expounded on in the next chapter.

Chapter 2

This chapter presents existing literature relevant to the development of the Patient-Doctor Management System for small rural clinics in the Philippines. It is divided into five sections: Conceptual Review of the core system concepts; Theoretical Framework applicable to the adoption and effectiveness of the system; Empirical Review showing real-world evidence from comparable settings; Research Gaps found in the literature; and the Evaluation of Research Methods.

2.1 Conceptual Review

2.1.1. Healthcare Information System and Electronic Medical Records

A Healthcare Information System (HIS) is a technological ecosystem designed to collect, store, and process medical data across global healthcare networks (Ngafeeson, 2014). It encompasses clinical, administrative, and financial operations. Under HIS is the Electronic Medical Records (EMR) system. EMR digitizes patient records within a single clinic, including medical history, diagnoses, and treatment plans, making them more accessible and secure (Hoory, 2022). It transforms fragmented patient records into a unified digital infrastructure, which prevents human errors and enhances healthcare delivery and efficiency. An effective EMR in a low-resource setting includes offline support, modular architecture, and interoperability (Sydzykova et al., 2017).

2.1.2 Role-Based Access Control

Role-Based Access Control (RBAC) is a data privacy standard for managing HIS authorization (de Carvalho Junior and Bandiera-Paiva, 2018). It groups users into roles, ensuring they can access only the information and perform functions within their scope of responsibility. However, real-world challenges were identified, including emergency access, context-awareness, and cloud interoperability. Notwithstanding, it is concluded from 17 out of the 20 reviewed articles that RBAC is not obsolete; it can evolve by supplementing it with modern security mechanisms like encryption.

2.1.3 Appointment and Billing Systems

A digital appointment system automates patient scheduling. It reduces conflicts and double bookings that typically arise in manual logbook processes, resulting in poor patient satisfaction and longer wait times (Cao, Wan, and Tu, 2011). However, widespread adoption requires addressing digital literacy and trust issues. Meanwhile, the billing system ensures that financial transactions, consultation fees, and pending balances are accurately recorded (Smith, 2025). Together, these modules are integral to a clinic information system, as it promotes transparency, lessen administrative burden, and improve clinic and patient interactions.

2.2 Theoretical Framework

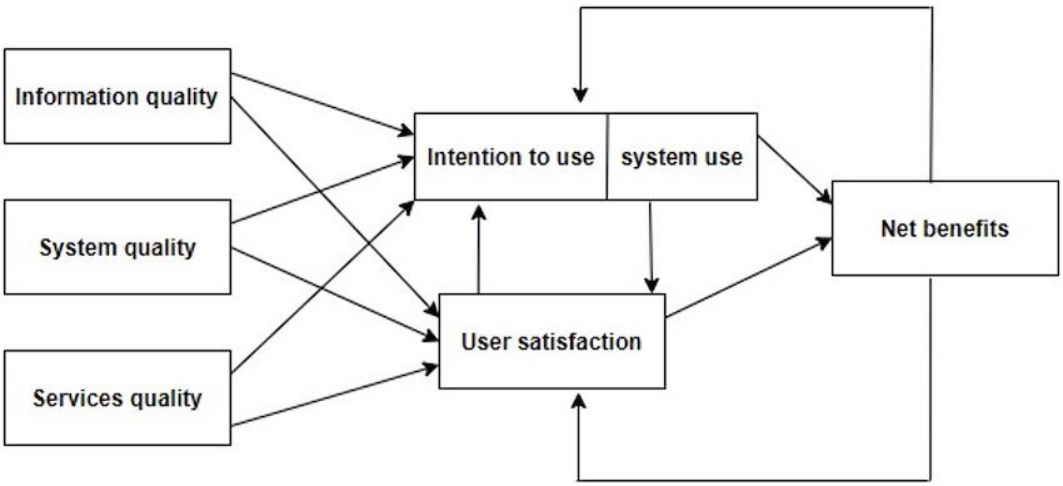


Figure 2.1: Information Systems Success Model (Delone & McLean, 2003)

The Information Systems Success Model identified that success categories such as System, Information, and Service qualities influence User Satisfaction and (Intention to) Use, which results in positive Net Benefits (Urbach and Muller, 2011). This model has been adapted to various information systems, including HIS, to evaluate the success of the implementation or whether it requires improvement.

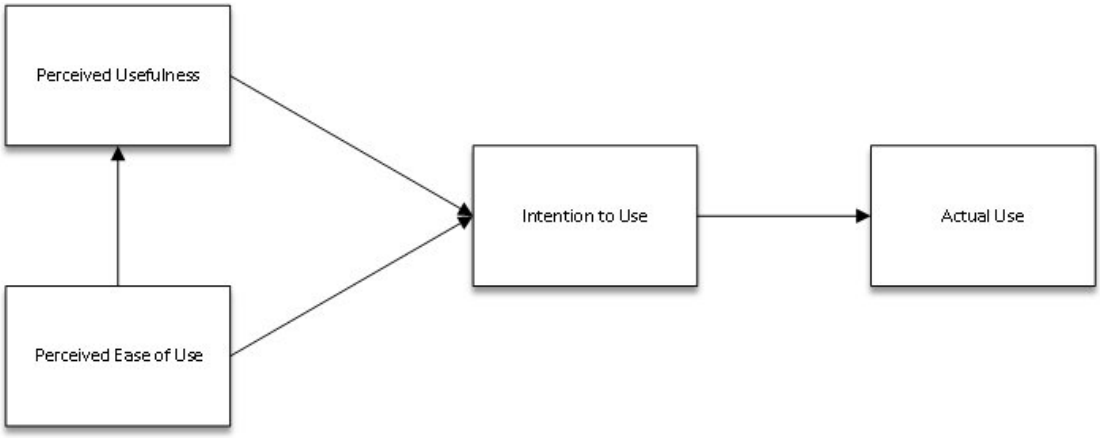


Figure 2.2: Technology Acceptance Model (Davis, 1989)

Another appropriate framework is the Technology Acceptance Model (TAM), which is designed to predict and explain why users accept or reject technology (Schorr, 2023). It explains that the intent to use a system is based on two perceptions: Perceived Usefulness (PU) and Perceived Ease of Use (PEOU). This creates a feedback loop for user retention. To synthesize, TAM serves as the rationale for the UI/UX and features, and the IS Success Model focuses on the back-end. Together, this ensures that the prototype is turned into a usable tool and is valuable to doctors and patients, while also meeting the acceptable standard of system quality essential for the healthcare environment.

2.3 Empirical Review

2.3.1 EMR Systems in the Philippines and Other Countries

In Nordic nations with advanced economies like Denmark, they have attained high digital maturity across hospitals, driven by consensus-based governance, public funding, robust digital infrastructures, and unique personal identification systems (CPR number) (Elisasen et al., 2025). On the contrary, developing countries such as Thailand and Malaysia encounter persistent issues, including fragmented implementation and limited infrastructure (Yi et al., 2024). In the Philippines, primary EMR initiatives like iClinicSys in rural areas have been partial and uneven (Macabasag et al., 2022). Another study shows that the EMR usage of six RHUs in Western Visayas was inconsistent, wherein most staff engage only with basic features. This emphasizes that small rural clinics in the Philippines remain significantly underserved amid the digital health revolution.

2.3.2 Offline versus Cloud-Based Systems in Rural Healthcare

Research shows that cloud-based systems offer scalability and centralized data access. However, it depends on a stable internet connection, which is a resource often unavailable in rural healthcare settings (Musunuri, 2025). Studies indicate that offline-first architectures, where data are stored locally and synchronize upon reconnection, are important for maintaining uninterrupted clinical operations during outages in low-connectivity environments (Eswar, 2025; Malanin, 2025). Examples of offline model architectures include Progressive Web Apps (PWAs) technologies and local storage, IndexedDB. Furthermore, a high acceptance (94%) was shown in Electronic Health Records across the Philippines rural sites, largely because the main adoption predictor is self-efficacy, and not connectivity (De Mesa et al., 2024). This highlights that users prefer simple and reliable systems regardless of network conditions.

2.4 Research Gaps

First, local studies focus predominantly on government-run facilities, which leaves small, rural clinics significantly unaddressed. Second, in comparable developing countries, available systems handle EMR, scheduling, and billing as separate tools rather than unifying them into a single platform. Lastly, research on access control design for single and offline-workstation clinic environments remains underexplored compared to large-scale hospital systems, where cloud-based has been the norm.

2.5 Evaluation of Methods

Drawing on 2018-2025 sources and limited field observations, the study produced a reliable basis for the system's design decisions, despite the scarcity and small samples. Evolutionary Prototyping iteratively refines the system based on user feedback (GeeksforGeeks, 2019), which determines the feasibility of offline functionality and modules. However, no live clinical trials; therefore, the real-world impact remains theoretical. Heuristic Evaluation (Moran & Gordon, 2023) allows the researcher to evaluate the system's usability against TAM's principles without medical participants, making it prone to bias. The research adviser is a patient who can give real-world, reasonable insights. Black-box testing will be executed, though it cannot fully mirror the unpredictability of an active rural clinic.

The literature further proves the need for an offline, role-based, integrated clinic management system for rural Philippine clinics. These findings significantly inform the first development phase of the project in the next chapter.

Chapter 3

This chapter presents the analytical foundation for the development and a detailed analysis of the requirements of PDMS. MoSCoW Prioritization explores the feature prioritization. Lastly, UML diagrams visualize both system functionality and component interaction.

3.1 Requirements Analysis

The PDMS requirements identified were informed by the literature reviewed in Chapter 2, and inspired by existing, comparable clinic management systems (BoostCRM, 2025; Nur Fi'llia Nugrahani, 2025). Below is the summary of functional requirements:

Functional Requirements

ID	Requirement	Role
FR-01	Register and log in via role-based credentials.	Both
FR-02	View and edit profile information.	
FR-03	Download invoices and Rx (PDF).	
FR-04	Book, reschedule, cancel, and view appointments.	Patient
FR-05	View prescription, diagnosis, and consultation notes from completed appointments.	
FR-06	View pending and paid payments.	
FR-07	Start (consultation), reschedule, cancel, and view appointments.	Doctor
FR-08	Search and view patients' medical records.	
FR-09	Log diagnoses, prescribe medications, and write laboratory requests.	
FR-10	View patients' unresolved balances, edit consultation fee, and verify payments.	

Below are the non-functional requirements that define how the system should perform regardless of the features.

Non-functional Requirements

Category	Requirement
Availability	Operate completely without internet connectivity.
Usability	Easy to use for users with little digital experience.
Performance	Run on low-specification computers.

Portability	Installed standalone with no external dependencies.
Data Integrity	Data is accurate, free from duplications and errors.
Security	Data access is based on user role. Data validation is enforced.

3.2 MoSCoW Prioritization

This technique classifies the features into four tiers, ensuring that development effort is directed to the most important functionality first, considering the project's time and resource constraints. The Must Have is the Minimum Viable Product (MVP), which are the features that fulfill the system's purpose. Should Have will be added once the MVP is stable. Could Have are stretch goals pursued if time permits, while Won't Have are the excluded features

PRIORITY	BOTH	PATIENT	DOCTOR
MUST HAVE	• Role-based registration and login. • View profile information. • View upcoming and completed appointments.	• Book an appointment.	• Start consultation. • Write a diagnosis. • Prescribe medications.
SHOULD HAVE	• Cancel and reschedule appointment. • View pending payments.	• View consultation details.	• Search patient and view records. • Confirm patient payments.
COULD HAVE	• Edit profile information. • Generate invoice and Rx.		• Update consultation fee. • Update laboratory request status.
WON'T HAVE	• Online payment gateway. • AI integration. • SMS/Email reminders.		

3.3 Use Case Diagram

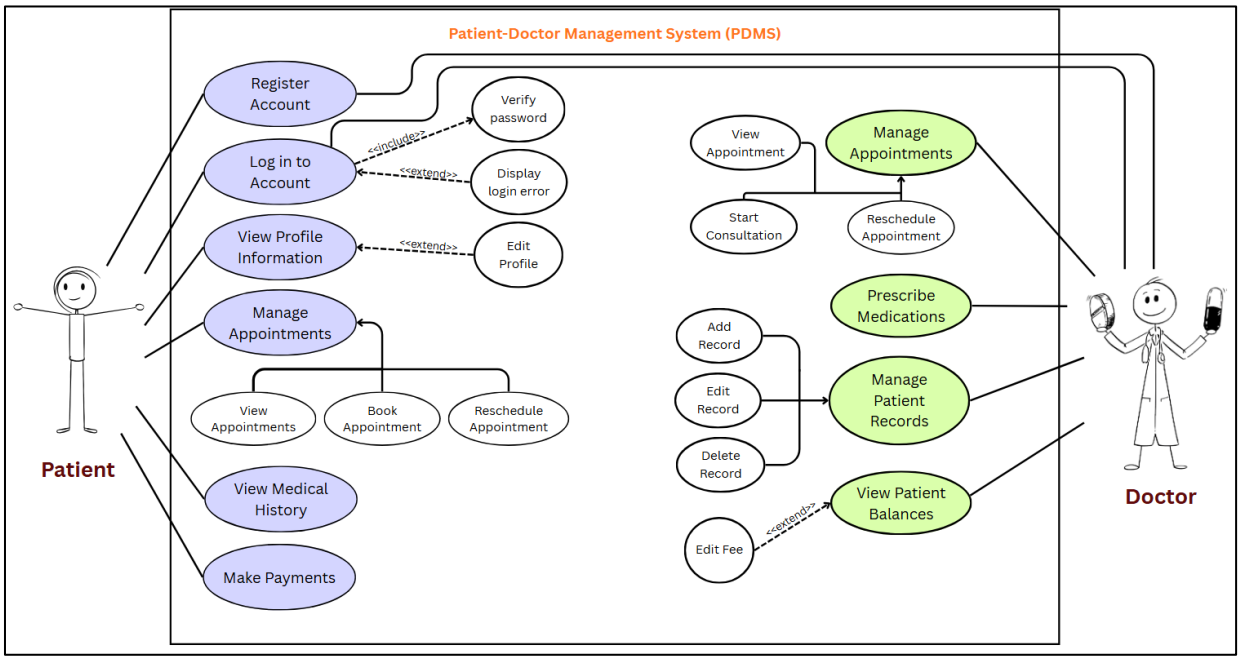


Figure 3.1: Initial Use Case Diagram

In the early stages, this diagram served as the structural blueprint, as it mapped actor interactions across different modules. As a visual learner, this diagramming process provided clarity on what components must be accomplished and is useful for target users, especially when starting from scratch.

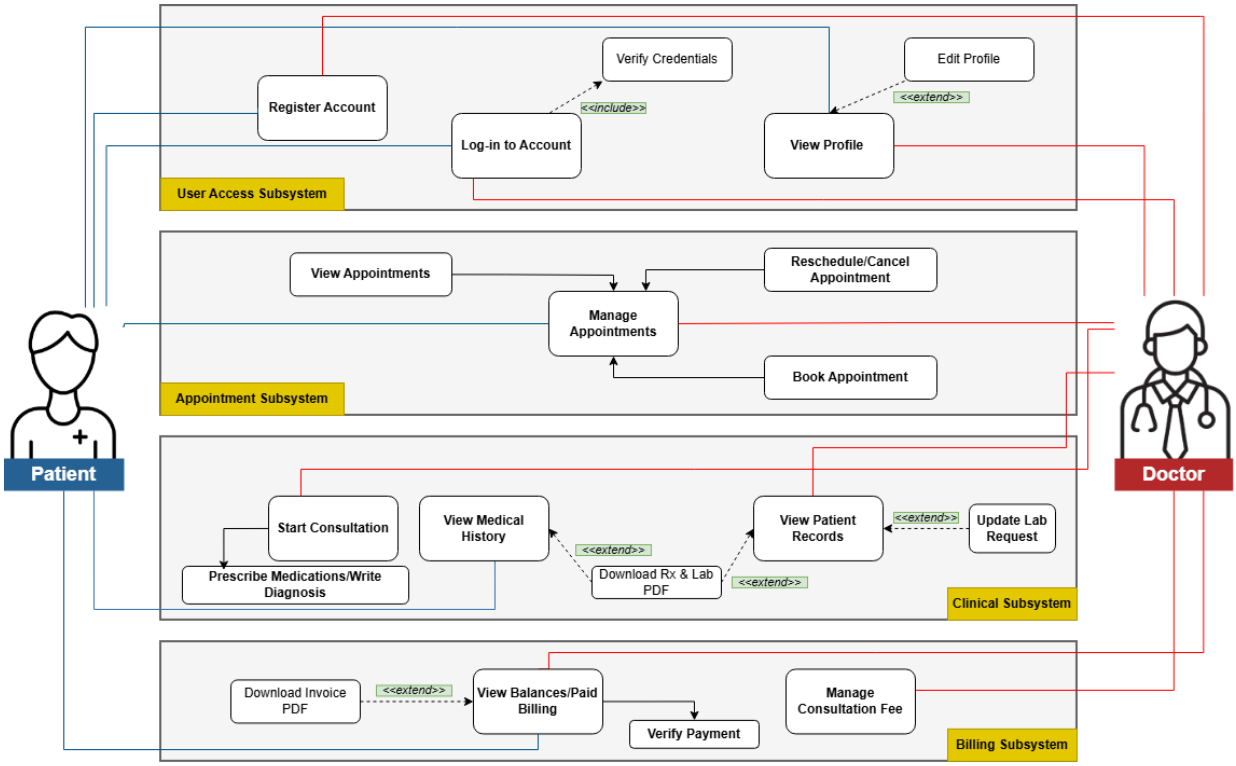
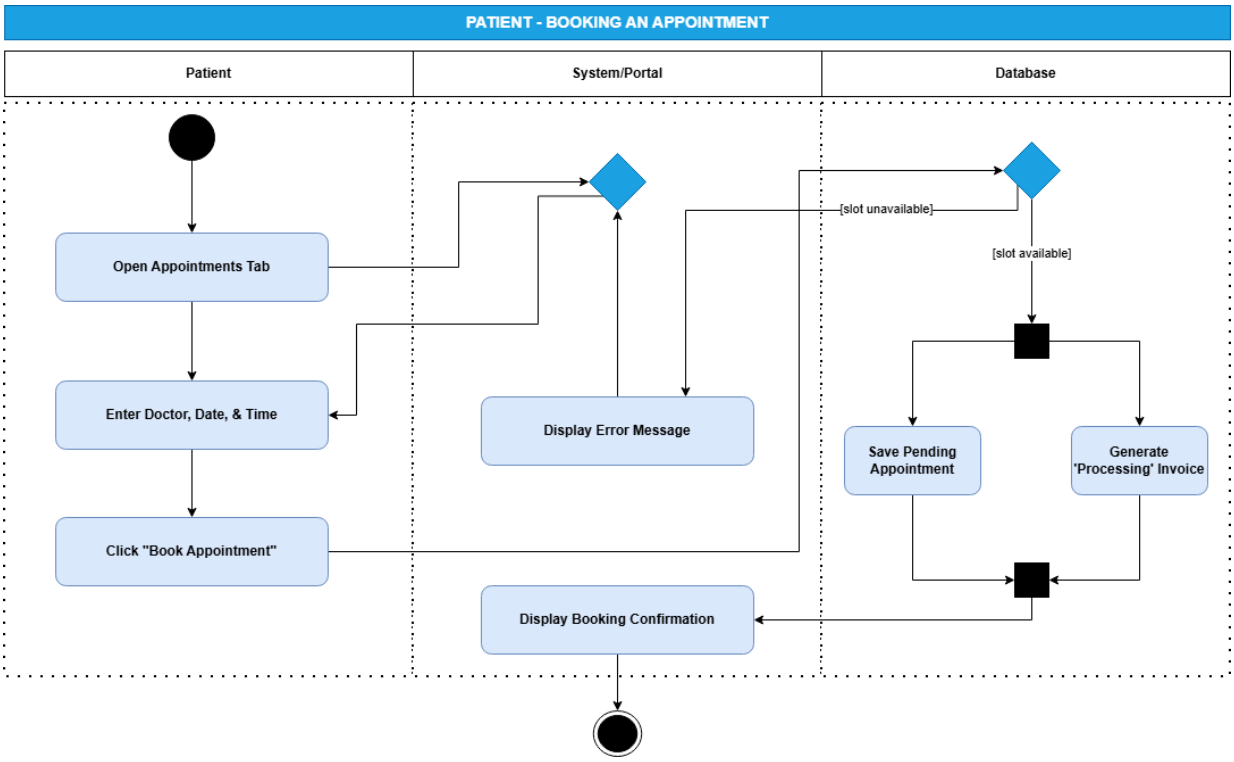
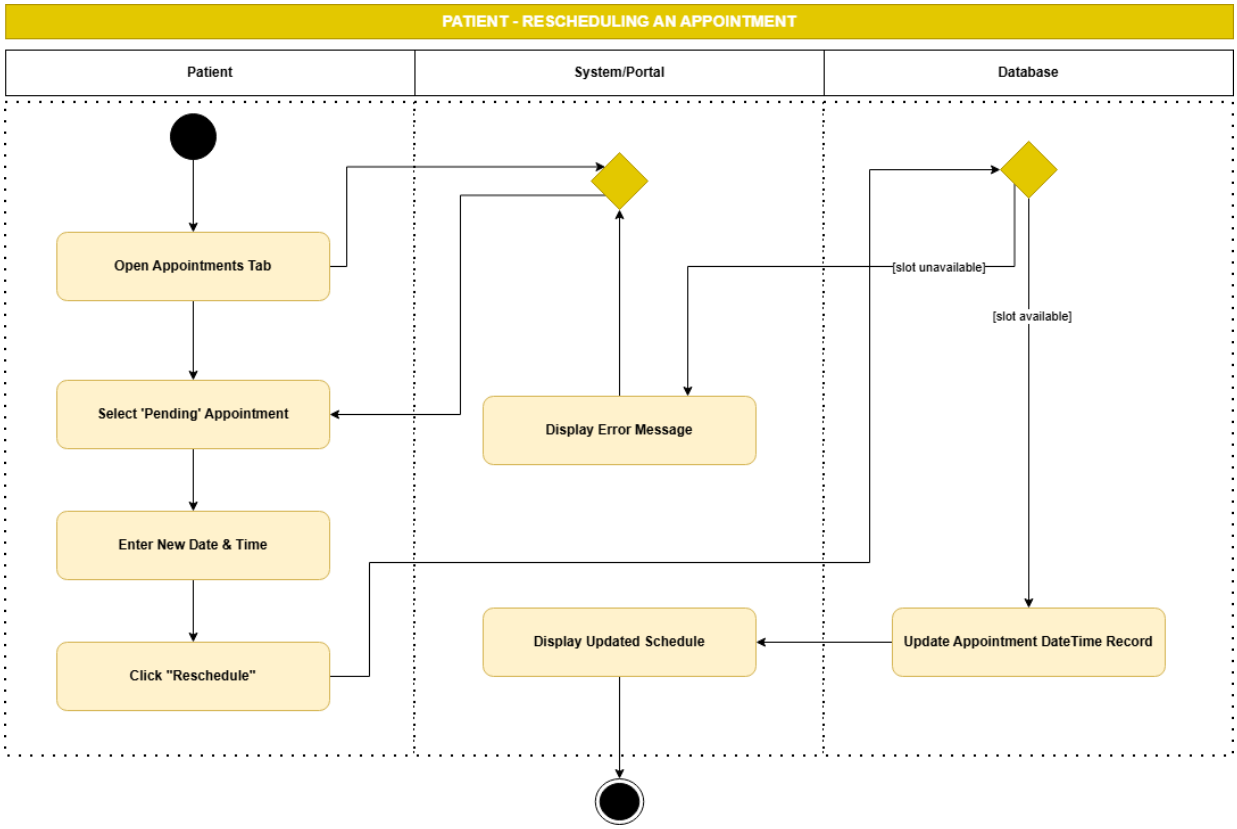


Figure 3.2: Finalized Use Case Diagram

As development progressed, the diagram underwent notable revisions. The use cases were expanded according to the system's implemented behavior and reorganized into subsystems, and relationships were refined and formalized compared to the initial diagram. Overall, it significantly informed the creation of wireframes, prevented scope creep, and has been a consistent reference for development. See *Appendix A* for use case diagram description.

3.4 Activity Diagrams





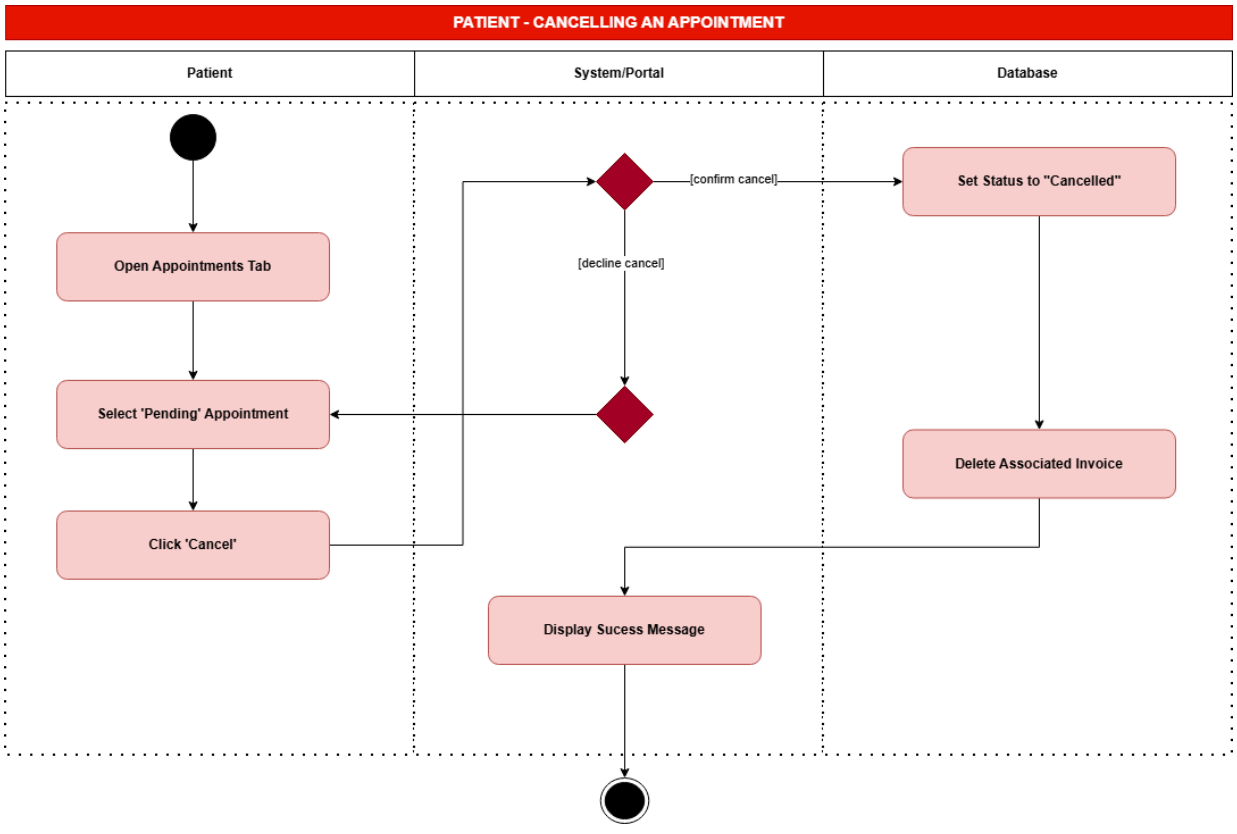


Figure 3.3: Patient Appointment Lifecyle: Booking, Rescheduling, and Cancellation Activity Diagram

The activity diagrams represent the appointment management process that possesses the same functionality for both user roles (except for the Booking functionality for the doctor). Iterative testing of each appointment functionality ensures that the designated database queries, such as INSERT, UPDATE, and DELETE statements, are working properly and that changes are reflected accurately in both portals. Pop-up messages (e.g., “Slot is booked.”, “Payment Required.”) were also added for conveying important information to the user, and are useful for enforcing data validation.

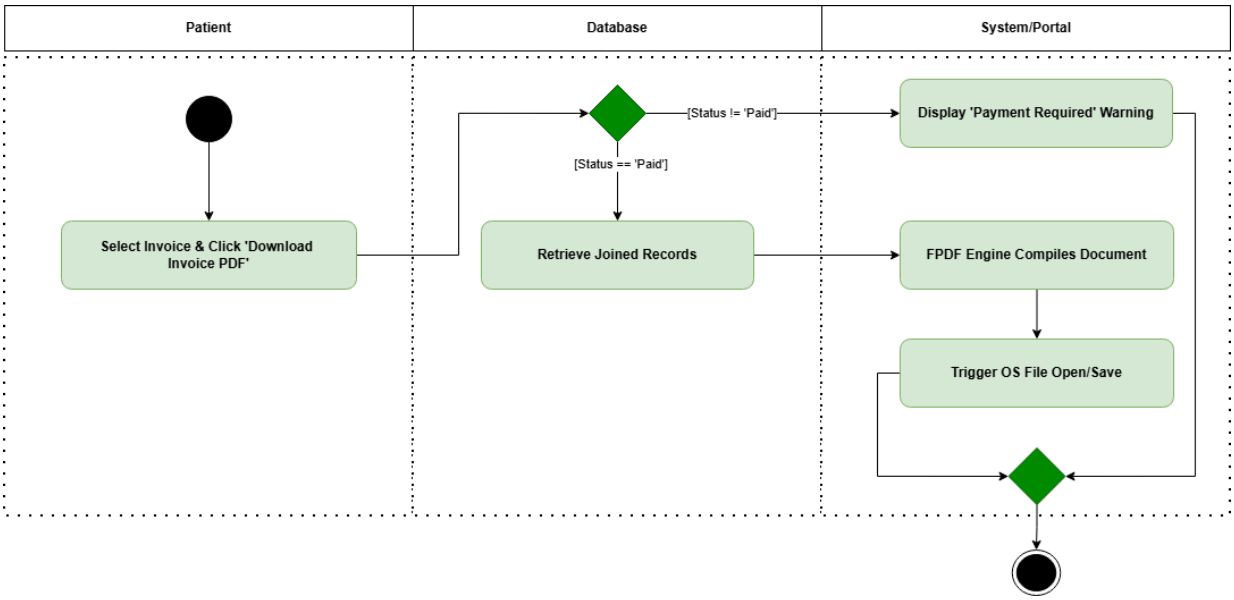


Figure 3.4: Invoice Resolution and PDF Generation Activity Diagram

This diagram represents the flow of downloading a billing invoice triggered by both the patient and the doctor. When the user clicks “Download Invoice PDF”, the system checks the payment status. If unpaid, a pop-up warning appears; if paid, records are retrieved from the local SQL Server. The FPDF library, a local rendering engine within Python, builds the document. The output is then persisted to disk with its designated file name. Lastly, the Operating System (OS) opens the file directly in the computer with any local PDF viewer installed in the browser. This entire PDF compilation architecture is 100% offline.

3.5 Sequence Diagrams

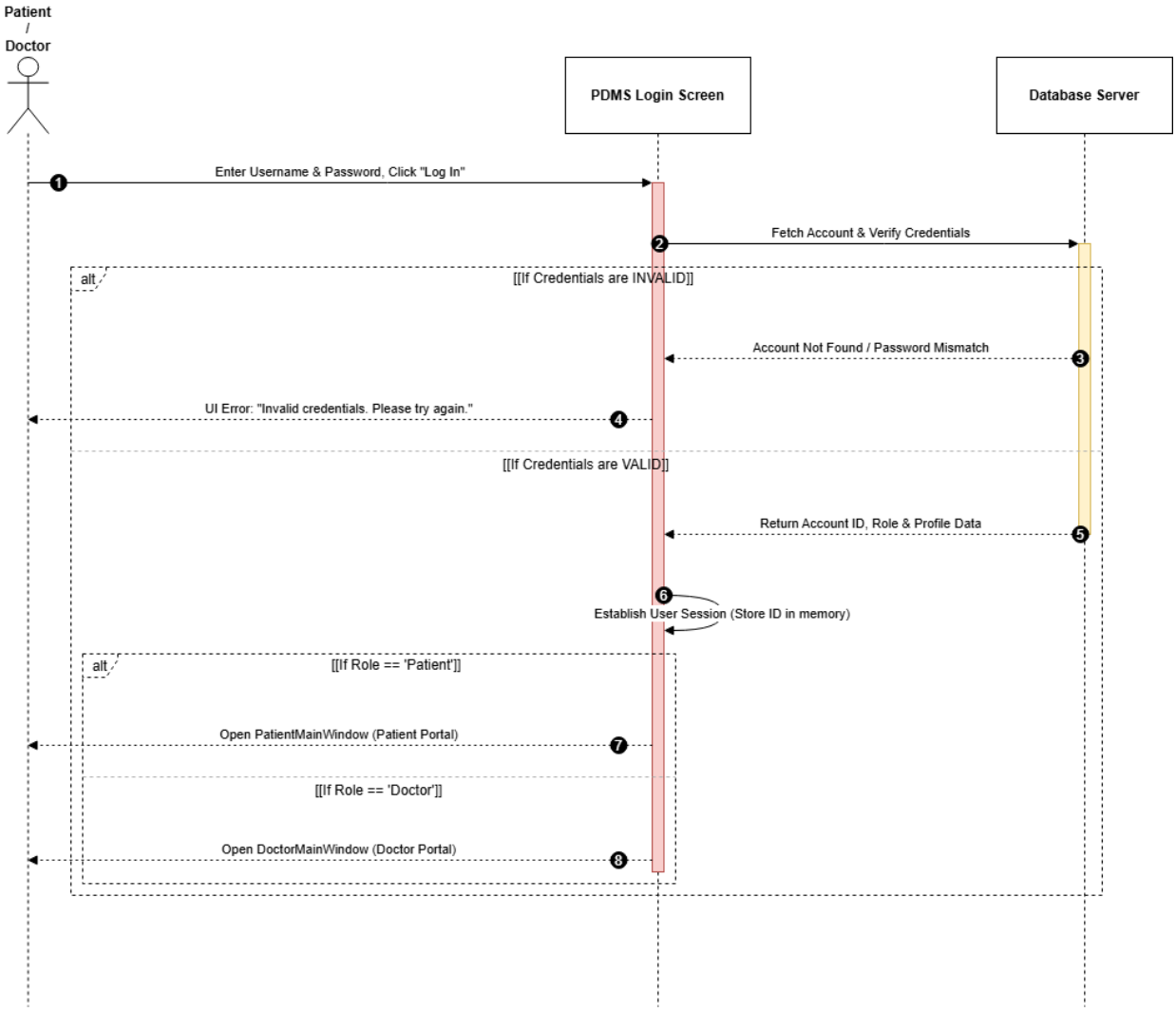


Figure 3.5: User Authentication and RBAC Routing Sequence Diagram

The creation of the sequence diagram was initially challenging, since significant independent study was done to understand UML message notation, return arrows, and activation boxes. It shows the back-and-forth interaction between the database and the system. However, this helped the researcher design and refine the RBAC logic at the early development stage, which eliminated ambiguity, and authentication if-else conditions were implemented correctly. It also clarified that role determination must happen at the database level, not user input.

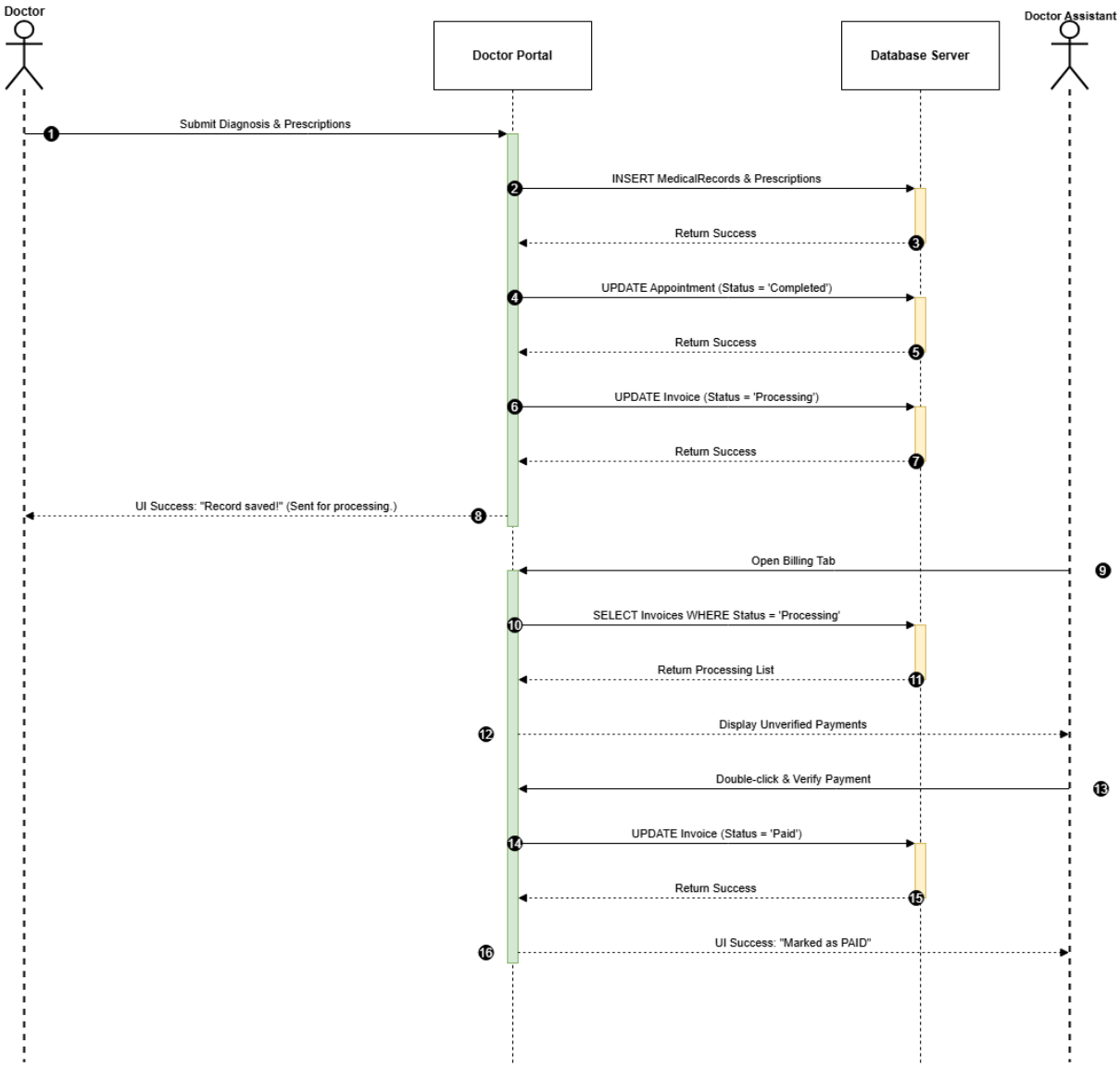


Figure 3.6: Clinical Consultation and Automated Billing State Sequence Diagram

The number of interdependent database operations triggered by a single user action makes this diagram more complex than the others. This visualization revealed that appointment status updates, invoice generation, and appointment completion must run in a coordinated sequence and not independently. The Doctor Assistant role is presented only as a hypothetical real-world scenario, in which an assistant in the clinic would manage billing tasks instead of the doctor in the portal itself. The implemented system does not include this role or designated portal, but it is acknowledged as a potential expansion, which will be further recommended in Chapter 7 as future work. For this project, the doctor will manage the billing tasks.

Although this phase has required independent research and meticulous production, it created the ultimate framework of the system. Each table and diagram resolved requirements ambiguities that would have been costly to address during the project's design and implementation. It helped break down the overwhelming, complex project into a simple one. It established development boundaries, and the diagrams collectively provided a reference that is understandable to stakeholders and developers for future work. The analytical work completed here directly shapes the design decision documented in Chapter 4.

Chapter 4

This chapter explains the primary design decisions and blueprint for building the Patient-Doctor Management System, and how they address the requirements stated in Chapter 3. It covers four areas: system and architectural, database, GUI, and algorithm designs.

4.1 System and Architectural Design

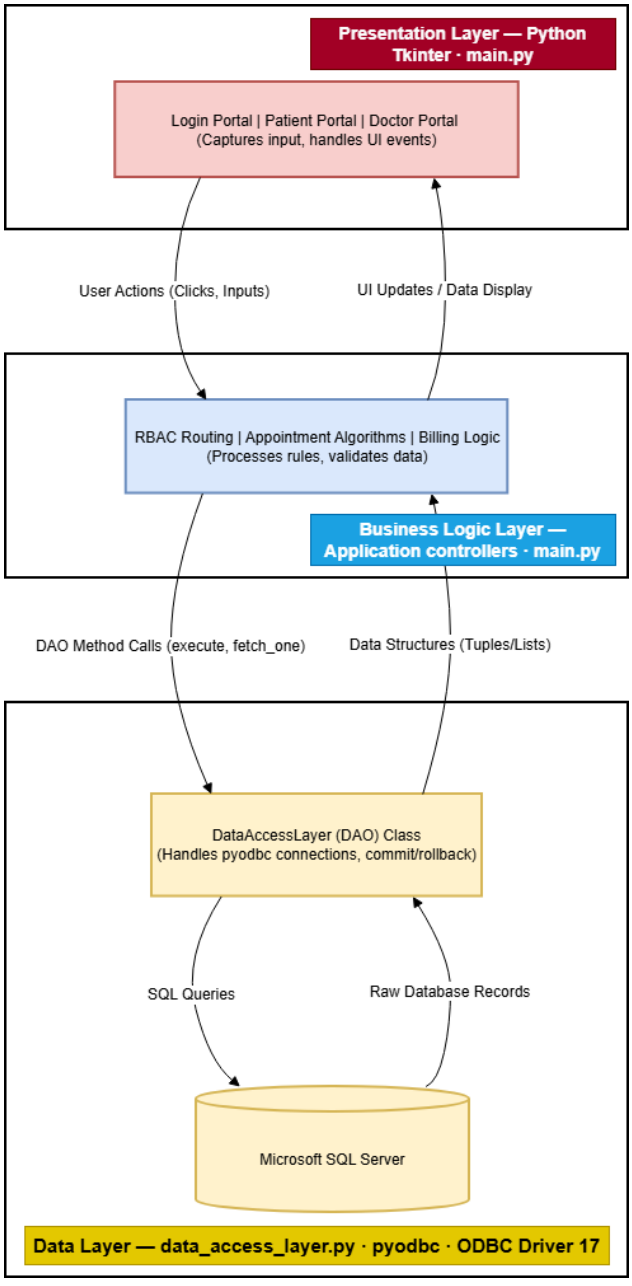


Figure 4.1: Three-Tier System Architecture

The architecture divides the Patient-Doctor Management System (PDMS) into three independent layers.

- Presentation Layer - This layer is responsible for all user-facing windows and event handling, such as the Login Portal, Patient Portal, and Doctor Portal.

- Business Logic Layer – Serves as the application controller. It contains the RBAC routing, algorithms, and input validation routines, ensuring that rules are processed and data is validated before passing to the database.
- Data Layer – This module manages *pyodbc* connections to the local Microsoft SQL Server and exposes standardized methods for executing queries, controlling transactions, and fetching results.

This architecture ensures that changes in one layer do not break the other. The communication flows downward, and the results return upward as tuples and lists for the UI to render.

```
CONNECTION_STRING = """
        Driver={ODBC Driver 17 for SQL
Server};Server=AmandaNitro\\SQLEXPRESS;
        Database=PDMS;
        Trusted_Connection=yes;
    """
```

Figure 4.2: Local Database Connection String

To satisfy the offline constraint, a monolithic desktop model was chosen over a web-based or cloud-hosted model. Since all clinics handle sensitive Protected Health Information (PHI), confining both the application and database on a single machine reduces exposure to external cybersecurity threats, such as unauthorized API access and remote data breaches, without the need for dedicated IT personnel and network infrastructure.

```
# snippet from main.py

dal = DataAccessLayer()
try:
    dal.open()

    # Parameterized execution (the '?' prevents SQL injection)
    user_id = dal.fetch_one(
        "INSERT INTO Users (Email, PasswordHash, Role, DateRegistered) "
        "OUTPUT INSERTED.UserID VALUES (?, ?, 'Patient', GETDATE())",
        email, password
    )

    patient_id = dal.fetch_one(
        "INSERT INTO Patients (UID, FirstName, LastName, Gender, ContactNumber) "
        "OUTPUT INSERTED.PatientID VALUES (?, ?, ?, ?, ?)",
        user_id, first_name, last_name, gender, contact_number
    )

    dal.execute(
        "INSERT INTO EmergencyContacts (PID, FullName, ContactNumber, Relationship)
"       "VALUES (?, ?, ?, ?)",
        patient_id, em_name, em_num, em_rel
    )

    # Atomic commit (it saves all 3 tables at once)
    dal.commit()

except Exception as err:
    # Atomic rollback (it prevents partial writes if ANY insert fails)
    dal.rollback()
    messagebox.showerror("Database Error", str(err))

finally:
    dal.close()
```

Figure 4.3: DAL Parameterized Execution and Atomic Rollback

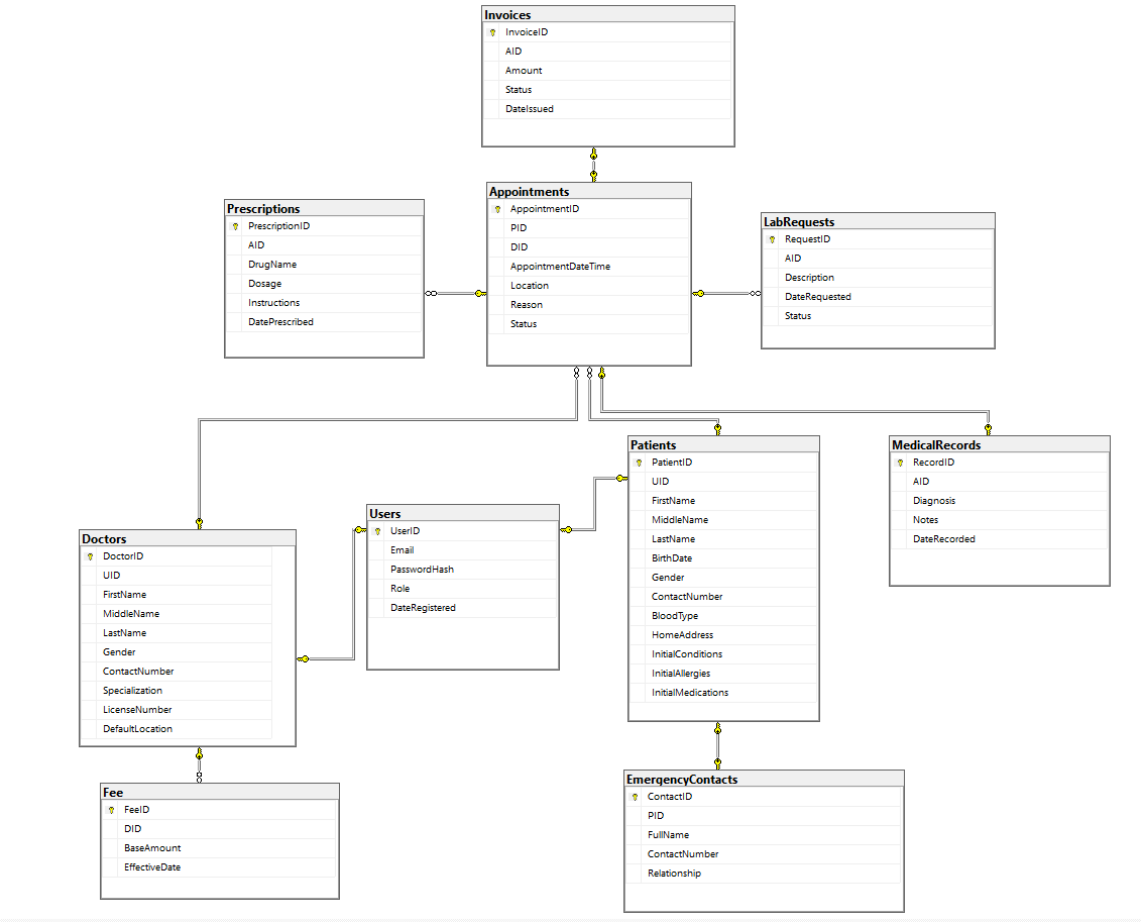
If raw SQL statements and connection strings are embedded into GUI event handlers, it would tightly bind the Presentation Layer to the Data Layer and break the principle of Separation

of Concerns (SOC) (Berezhna, 2025). This led to another crucial design decision — implementation of the Data Access Object (DAO) pattern through the *DataAccessLayer*.class. Data access logic is isolated neatly from the rest of the application to handle communication between the program and database (GeeksforGeeks, 2017). All database queries and transactions go through standardized methods: *execute(), execute_query()*, *fetch_one()*, *and commit()/rollback().* This abstraction ensures that the data layer is independently testable and replaceable, as the UI is unaware of the underlying database engine.

Furthermore, DAL centers on transaction management, where most multi-table write operations enforce *try-except-finally* blocks, as it is treated as a single atomic unit. It affirms that any failures trigger an atomic rollback that prevents partial writes and maintains ACID (Atomicity, Consistency, Isolation, Durability) compliance. It is worth mentioning that this architecture is a design intention; during implementation, some business logic may need to reside closer to the UI layer due to Tkinter's event-driven constraints that make separation impractical.

The object-oriented programming paradigm was applied by encapsulating each functional area of the system within a designated class (e.g., *LoginWindow*, *PatientRegistrationWindow*,). The classes contain respective *__init__* constructor method for instance variable binding and widget initialization. Instance variables like *patient_data{}* and *doctor_data{}* are scoped within their respective portal objects, which avoids potential cross-class data leakage. Modal helpers like *RescheduleWindow*, *CalendarPopup*, and *EditProfileWindow* are independently reusable and importable components, which show OOP's benefit of modularity. Lastly, the *open_main_window()* acts as a routing controller that connects the login process to the correct portal by instantiating the appropriate class after authentication.

4.2 Database Design



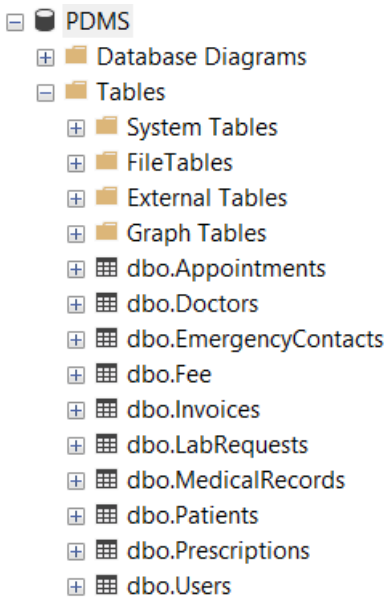


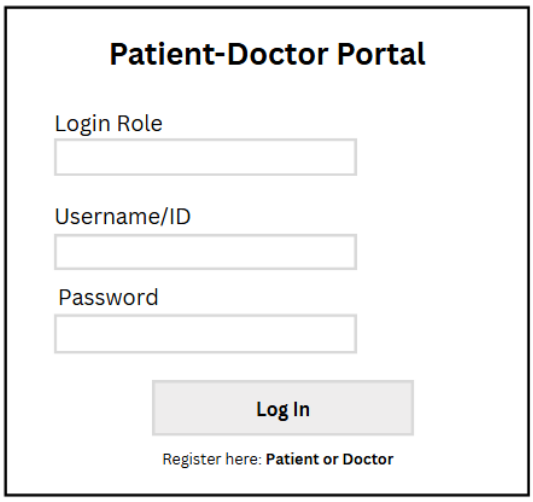
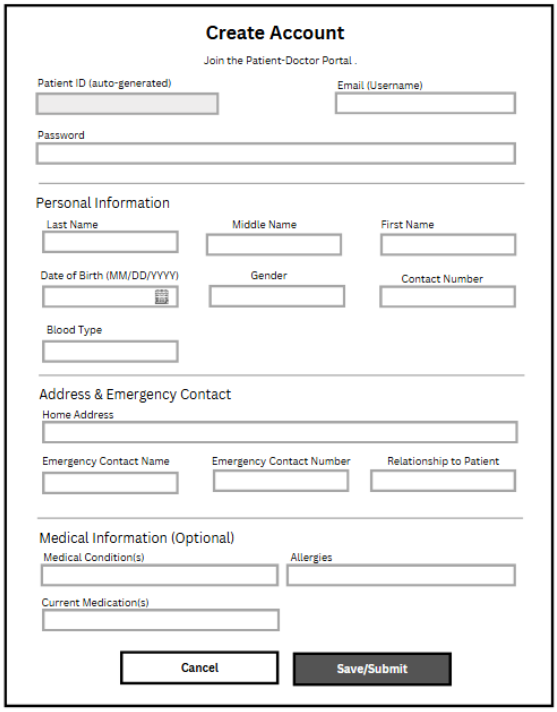
Figure 4.4 Entity-Relationship Diagram

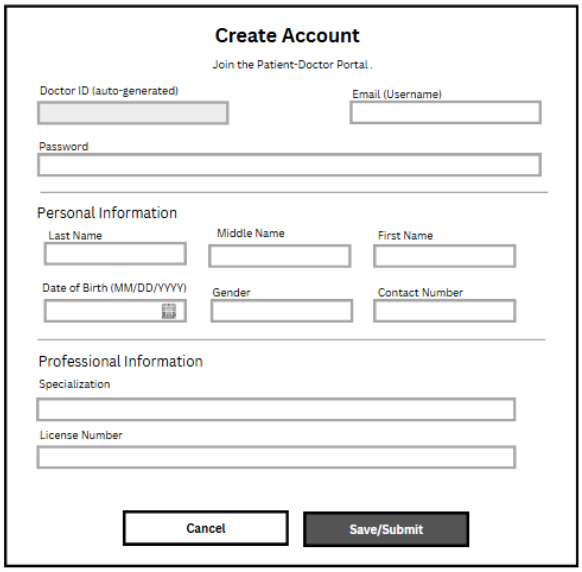
The relational database schema was designed to Third Normal Form (3NF) to remove redundancies, avoid update anomalies, and maintain absolute referential integrity through PK, FK, and CHECK constraints. All tables use primary IDENTITY(1,1) for the PKs, so it is auto-incremented, rather than manual ID assignment. A notable design decision was using the *Users* table as the root authentication entity. It enforces RBAC through a CHECK constraint that restricts roles to either 'Doctor' or 'Patient'. From this root, the schema integrates a sub-typing architecture. The *Patients* and *Doctors* tables are linked through a *UID* (or User ID) foreign key, which means that a single authentication identity cannot be linked to several clinic profiles. At login, the role value determines which portal class is instantiated, so only the designated interface and widgets are constructed in memory. This supports the Principle of Least Privilege (Yasar and Gillis, 2025).

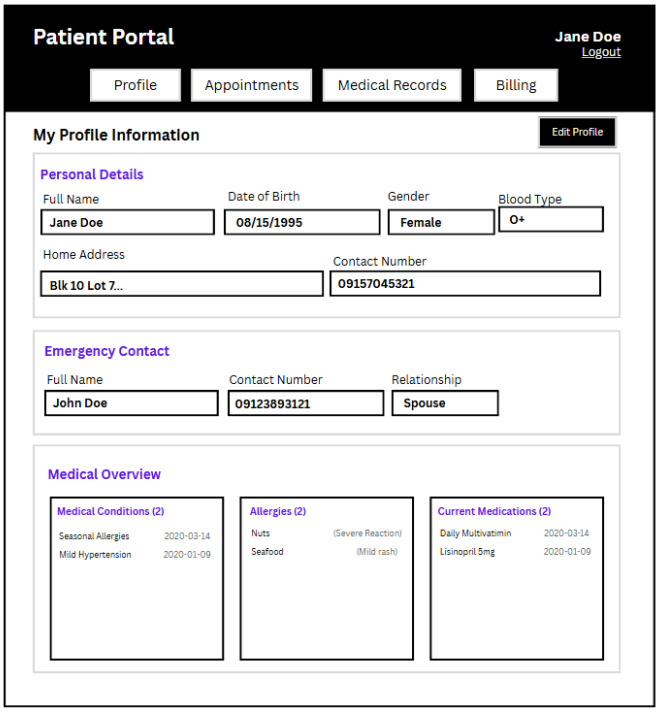
Meanwhile, the *Appointments* table functions as the core junction entity that resolves the many-to-many relationships. It uses two independent FK keys that link a particular patient and doctor profiles to a unique *AppointmentID*. Once generated, all clinical and financial entities are relationally bound to it. For example, the *MedicalRecords.AID* and *Invoices.AID* incorporates a one-to-one relationship, guaranteeing that one appointment links to one invoice and one medical record, which is a prerequisite for the upsert process in the *submit_prescription()* function. Conversely, the *Prescriptions* and *LabRequests* tables enforce a one-to-many relationship, which allows a doctor to issue multiple medications or laboratory requests in a single visit. See *Appendix B* for Data Dictionary.

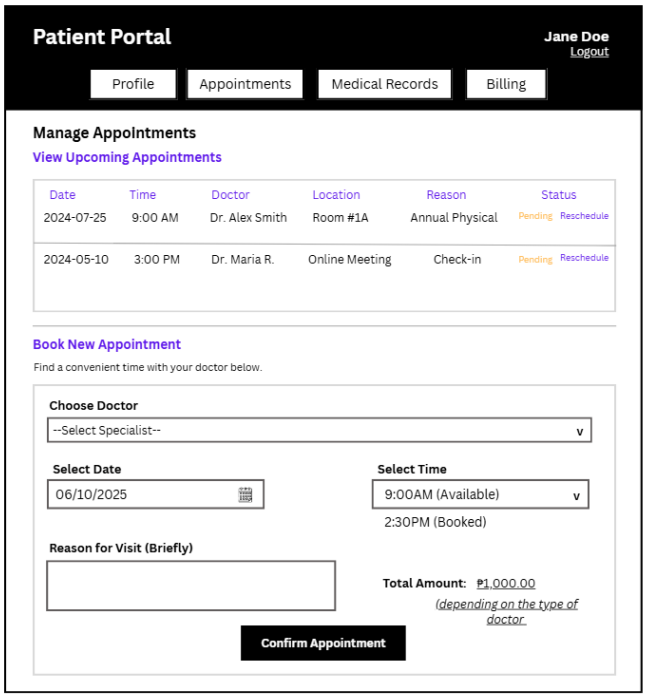
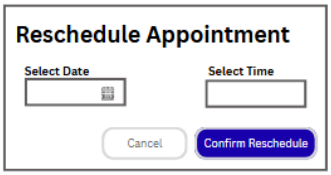
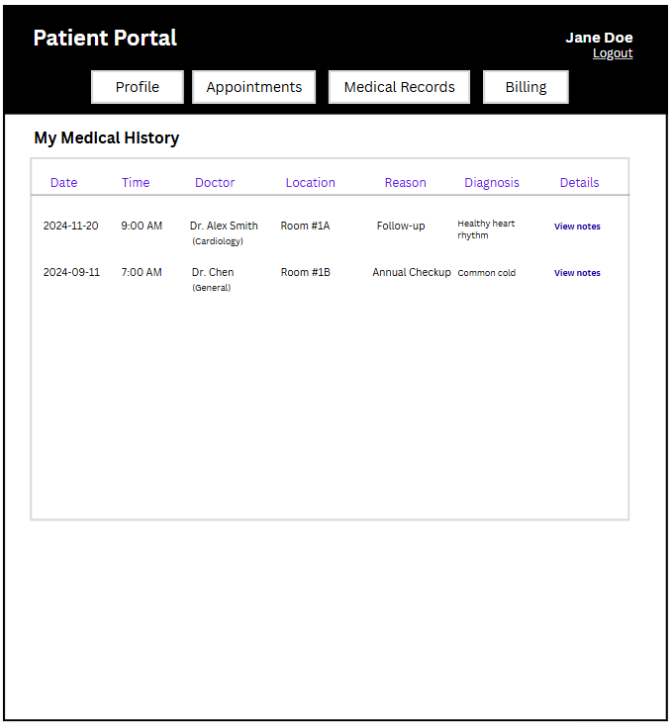
4.3 GUI and Algorithm Design

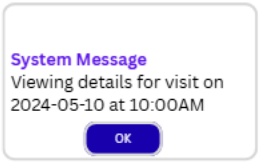
High-fidelity wireframes were created alongside the initial UML diagrams in October 2025, using Canva. These wireframes help visualize all the screen layouts, user flows, and tab structures before writing the code.

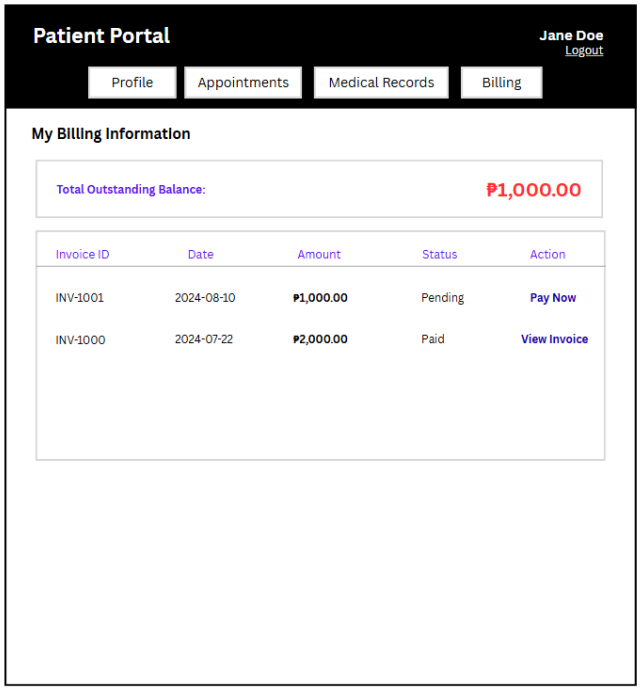
Login and Registration Windows		
Window	**Wireframe**	**Algorithm**
Login		1. User selects Role. 2. Enters credentials. 3. Clicks Log In. 4. System validates the fields. 5. System queries the *Users* table. 6. If a match is found, the user is routed to either the Patient or the Doctor portal. 7. If no match, the system displays an error.
Patient Registration		Both roles: 1. User fills in all required fields. 2. System validates the fields: 3. If successful, profile information rows are inserted. 4. Redirect back to the Login Window.

Doctor Registration		

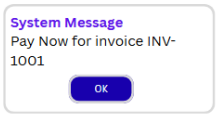
Patient Portal		
Profile		1. Click ‘Edit Profile’. 2. System loads editable personal details. 3. Updates information. 4. System validates: • If successful, overwrite the existing records. • If unsuccessful, show an error. 5. Return to the Profile tab.

Appointments	 	1. Select a doctor, date, and timeslot. 2. System retrieves existing bookings and filters available timeslots. 3. Choose timeslot. 4. Enters reason. 5. Confirm appointment. 6. Saves new appointment.
Medical Records		Completed visits are displayed. Placeholder tabs were initially planned.

<table>
<tr>
<td></td>
<td>
View notes
When clicked, it shows a small window popup:


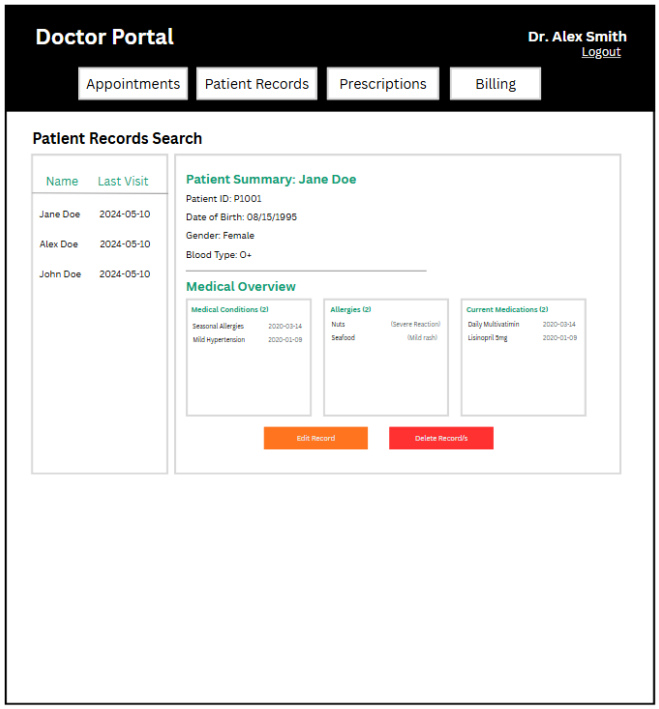
</td>
<td></td>
</tr>
<tr>
<td>Billing</td>
<td>


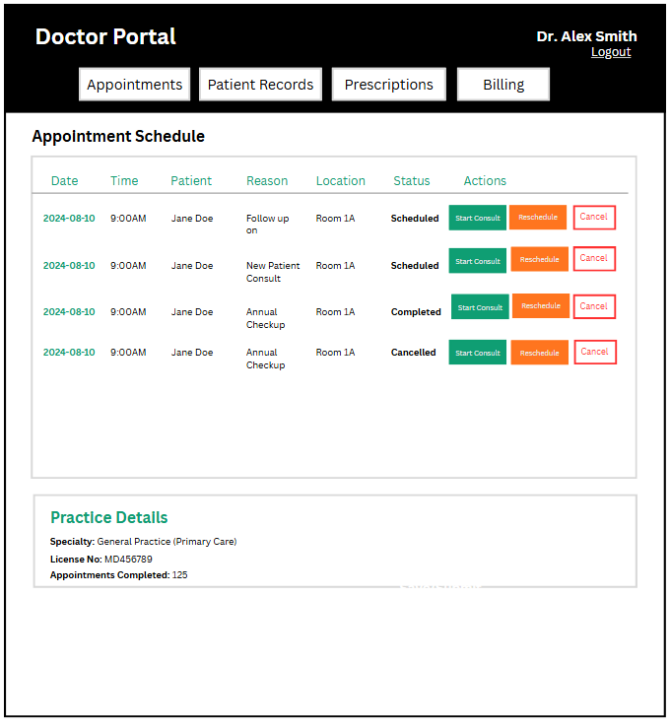
View PDF
When clicked, it shows a small window popup:

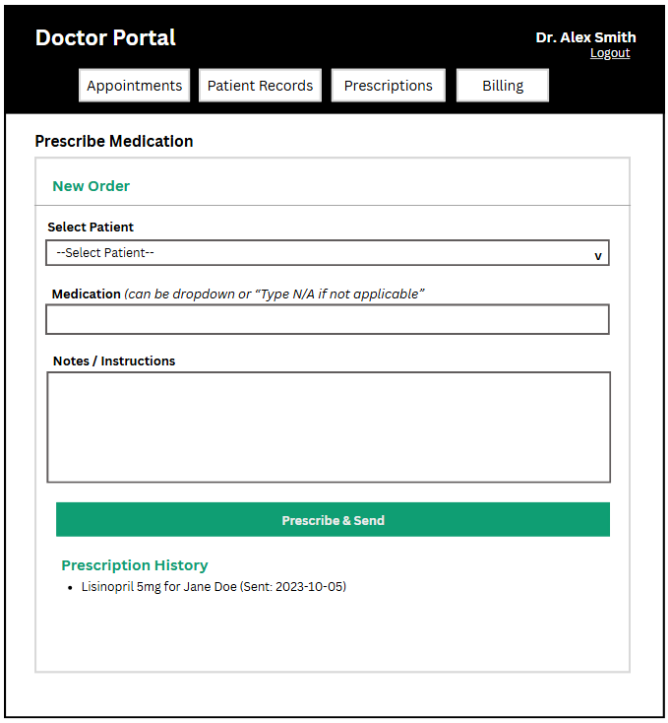
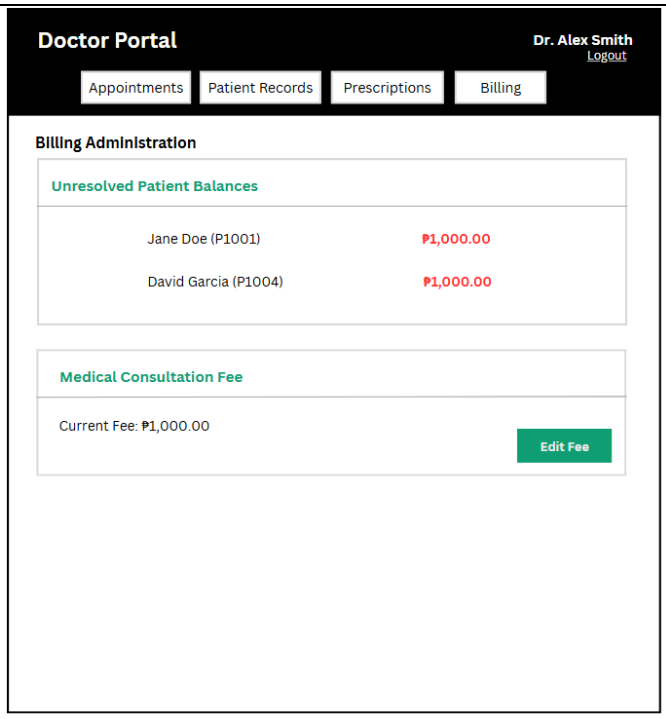

Pay Now
When clicked, it shows a small window popup:


</td>
<td>
1. Click ‘Pay Now’.
2. A confirmation pop-up appears.
3. On confirmation, the system updates the ‘Pending’ invoice to ‘Paid’ status.
4. Recalculates outstanding balance.

‘Pay Now’ button was removed; a new payment loop was introduced.
</td>
</tr>
</table>

<table>
<tr><th colspan="3">Doctor Portal</th></tr>
<tr><td>Patient Records</td><td> 
</td><td>Displays patient list and summary.

'Edit/Delete records were omitted due to their added complexity and potential scope creep.</td></tr>
<tr><td>Appointments</td><td> 
</td><td>1. Clicks ‘Start Consult’.
2. System directs to the Prescriptions tab with the patient pre-loaded.
3. Enters fields.
4. Click ‘Prescribe & Send.’
5. System runs one transaction:
• Saves the medical record.
• ‘Completed’ appointment status.</td></tr>
</table>

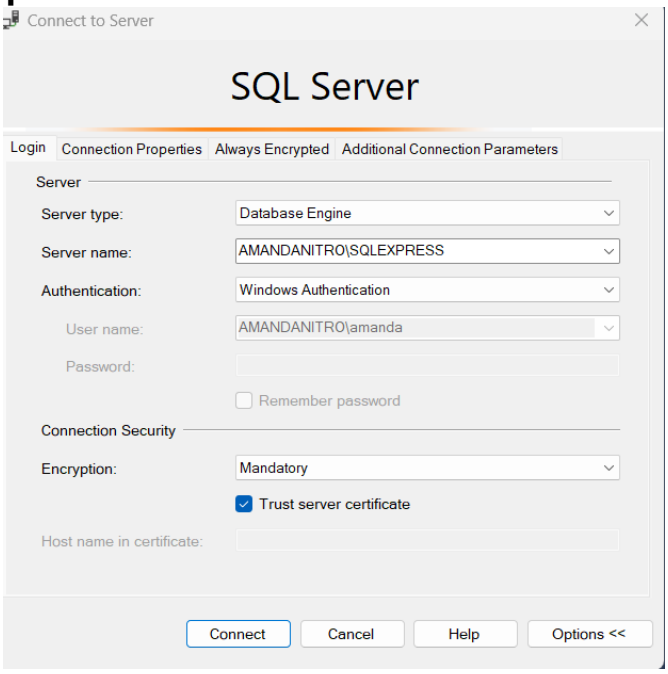
Prescriptions		• 'Pending' invoice status.
Billing		1. Clicks 'Edit Fee'. 2. Enter new amount. 3. Click Confirm. 4. System inserts a new *Fee* row. 5. Existing invoices are unaffected.

This chapter successfully explained the three design pillars: three-tier architecture, normalized relational database schema, and the high-fidelity wireframe. The wireframes weren't entirely identical to the finished GUI, due to widget positioning challenges; nonetheless, they provided structure to the program, all of which are documented in Chapter 5.

Chapter 5

This chapter narrates the system implementation, the challenges faced, and the solutions taken. Development spanned roughly five months, from October 2025 to May 2026, excluding major holiday breaks.

5.1 Database Population




```
--Users
INSERT INTO dbo.Users (Email, PasswordHash, Role)
VALUES
    ('dr.luna@pdms.ph', 'CardioDoc@2026!', 'Doctor'),
    ('dr.delmundo@pdms.ph', 'PediaDoc@2026!', 'Doctor'),
    ('dr.rizal@pdms.ph', 'GenMedDoc@2026!', 'Doctor'),
    ('dr.valdez@pdms.ph', 'DermaCare@2026!', 'Doctor'),
    ('dr.tolentino@pdms.ph', 'PsychMind@2026!', 'Doctor'),
    ('juan.delacruz@email.ph', 'Patient@2026!', 'Patient'),
    ('clara.santos@email.ph', 'Patient@2026!', 'Patient'),
    ('pedro.penduko@email.ph', 'Patient@2026!', 'Patient'),
    ('gabriela.silang@email.ph', 'Patient@2026!', 'Patient'),
    ('miguel.aguinaldo@email.ph', 'Patient@2026!', 'Patient');

--Doctors
INSERT INTO dbo.Doctors (UID, FirstName, MiddleName, LastName, Gender, ContactNumber, Specialization, LicenseNumber, DefaultLocation)
VALUES
    (1, 'Antonio', 'Reyes', 'Luna', 'M', '09171000001', 'Cardiology', 'DR-2026-8001', 'Room 301 - Heart Center'),
    (2, 'Fe', 'Villanueva', 'Del Mundo', 'F', '09182000002', 'Pediatrics', 'DR-2026-8002', 'Room 105 - Pedia Ward'),
    (3, 'Jose', 'Mercado', 'Rizal', 'M', '09193000003', 'General Medicine', 'DR-2026-8003', 'General Clinic'),
    (4, 'Vicki', 'Santos', 'Valdez', 'F', '09204000004', 'Dermatology', 'DR-2026-8004', 'Skin Clinic'),
    (5, 'Arturo', 'Garcia', 'Tolentino', 'M', '09215000005', 'Psychiatry', 'DR-2026-8005', 'Room 402 - Wellness Center');

--Patients
INSERT INTO dbo.Patients (UID, FirstName, MiddleName, LastName, BirthDate, Gender, ContactNumber, BloodType, HomeAddress, InitialConditions, InitialAllergies, InitialMedications)
VALUES
    (6, 'Juan', 'Bautista', 'Dela Cruz', '1980-05-15', 'M', '09177771111', 'O+', 'Blk 1 Lot 2, Salinas, Bacoor City', 'Hypertension', 'None', 'Losartan 50mg'),
    (7, 'Clara', 'Gomez', 'Santos', '1995-12-01', 'F', '09188882222', 'A-', 'Aguinaldo Highway, Dasmariñas City', 'Asthma', 'Dust mites, Penicillin', 'Salbutamol'),
    (8, 'Pedro', 'Roxas', 'Penduko', '1990-08-21', 'M', '09199993333', 'B+', 'Molino Blvd, Bacoor City', 'None', 'None', 'None'),
    (9, 'Gabriela', 'Diego', 'Silang', '2001-03-08', 'F', '09200004444', 'AB+', 'Bayan Luma, Imus City', 'PCOS', 'Seafood', 'None'),
    (10, 'Miguel', 'Cruz', 'Aguinaldo', '2018-11-30', 'M', '09211115555', 'O-', 'Centennial Road, Kawit', 'None', 'Peanuts', 'Vitamin C');
```

Figure 5.1: Users' Insertion Excerpt

Insertion of data was done through the registration windows to verify constraints and validation rules. However, as the system expanded, this method became impractical and time-consuming. Thus, a T-SQL seed script was written *(SCOPE_IDENTITY())*.

```
SELECT * FROM dbo.Patients;
SELECT * FROM dbo.Doctors;
SELECT * FROM dbo.Appointments;
SELECT * FROM dbo.Invoices;
SELECT * FROM dbo.EmergencyContacts;
SELECT * FROM dbo.Fee;
SELECT * FROM dbo.MedicalRecords;
```

100 %

Results Messages

	UserID	Email	PasswordHash	Role	DateRegistered
1	1	dr.luna@pdms.ph	CardioDoc@2026!	Doctor	2026-05-26 18:03:13.3366667
2	2	dr.delmundo@pdms.ph	PediaDoc@2026!	Doctor	2026-05-26 18:03:13.3366667
3	3	dr.rizal@pdms.ph	GenMedDoc@2026!	Doctor	2026-05-26 18:03:13.3366667
4	4	dr.valdez@pdms.ph	DermaCare@2026!	Doctor	2026-05-26 18:03:13.3366667
5	5	dr.tolentino@pdms.ph	PsychMind@2026!	Doctor	2026-05-26 18:03:13.3366667
6	6	juan.delacruz@email.ph	Patient@2026!	Patient	2026-05-26 18:03:13.3366667
7	7	clara.santos@email.ph	Patient@2026!	Patient	2026-05-26 18:03:13.3366667
8	8	pedro.penduko@email.ph	Patient@2026!	Patient	2026-05-26 18:03:13.3366667

	PatientID	UID	FirstName	MiddleName	LastName	BirthDate	Gender	ContactNumber	BloodType	HomeAddress	InitialConditions	InitialAllergies	InitialMedications
1	1	6	Juan	Bautista	Dela Cruz	1980-05-15	M	09177771111	O+	Blk 1 Lot 2, Salinas, Bacoor City	Hypertension	None	Losartan 50mg
2	2	7	Clara	Gomez	Santos	1995-12-01	F	09188882222	A-	Aguinaldo Highway, Dasmariñas City	Asthma	Dust mites, Penicillin	Salbutamol
3	3	8	Pedro	Roxas	Penduko	1990-08-21	M	09199993333	B+	Molino Blvd, Bacoor City	None	None	None
4	4	9	Gabriela	Diego	Silang	2001-03-08	F	09200004444	AB+	Bayan Luma, Imus City	PCOS	Seafood	None
5	5	10	Miguel	Cruz	Aguinaldo	2018-11-30	M	09211115555	O-	Centennial Road, Kawit	None	Peanuts	Vitamin C

	DoctorID	UID	FirstName	MiddleName	LastName	Gender	ContactNumber	Specialization	LicenseNumber	DefaultLocation
1	1	1	Antonio	Reyes	Luna	M	09171000001	Cardiology	DR-2026-8001	Room 301 - Heart Center
2	2	2	Fe	Villanueva	Del Mundo	F	09182000002	Pediatrics	DR-2026-8002	Room 105 - Pedia Ward
3	3	3	Jose	Mercado	Rizal	M	09193000003	General Medicine	DR-2026-8003	General Clinic
4	4	4	Vicki	Santos	Valdez	F	09204000004	Dermatology	DR-2026-8004	Skin Clinic
5	5	5	Arturo	Garcia	Tolentino	M	09215000005	Psychiatry	DR-2026-8005	Room 402 - Wellness Center

	AppointmentID	PID	DID	AppointmentDateTime	Location	Reason	Status
1	1	1	1	2026-04-15 09:00:00.0000000	Room 301 - Heart Center	Chest pain and routine BP check	Completed
2	2	2	3	2026-05-02 14:00:00.0000000	General Clinic	Fever, cough, and body aches for 3 days	Completed
3	3	4	4	2026-06-10 10:00:00.0000000	Skin Clinic	Severe acne breakout consultation	Pending
4	4	3	3	2026-06-15 13:00:00.0000000	General Clinic	Annual Physical Exam for pre-employ...	Resched...
5	5	1	5	2026-05-10 15:00:00.0000000	Room 402 - Wellness C...	Work-related stress management	Cancelled
6	6	5	2	2026-04-20 11:00:00.0000000	Room 105 - Pedia Ward	Routine vaccination and weight check	Completed
7	7	1	1	2026-06-25 09:00:00.0000000	Room 301 - Heart Center	Follow-up on Amlodipine effectiveness	Pending

	InvoiceID	AID	Amount	Status	DateIssued
1	1	1	1500.00	Paid	2026-05-26 18:03:13.4066667
2	2	2	500.00	Paid	2026-05-26 18:03:13.4066667
3	3	6	800.00	Paid	2026-05-26 18:03:13.4066667
4	4	3	1000.00	Pen...	2026-05-26 18:03:13.4066667

	ContactID	PID	FullName	ContactNumber	Relationship
1	1	1	Maria Dela Cruz	09171234567	Wife
2	2	2	Roberto Santos	09181234567	Father
3	3	3	Anna Penduko	09191234567	Sister
4	4	4	Mario Silang	09201234567	Brother
5	5	5	Elena Aguinaldo	09211234567	Mother

	FeeID	DID	BaseAmount	EffectiveDate
1	1	1	1500.00	2026-05-26 18:03:13.3766667
2	2	2	800.00	2026-05-26 18:03:13.3766667
3	3	3	500.00	2026-05-26 18:03:13.3766667
4	4	4	1000.00	2026-05-26 18:03:13.3766667
5	5	5	2000.00	2026-05-26 18:03:13.3766667

	RecordID	AID	Diagnosis	Notes	DateRecorded
1	1	1	Mild Angina	Patient advised to rest, monitor BP daily, and a...	2026-04-15 10:00:00.0000000
2	2	2	Viral Pharyngitis	Throat is visibly inflamed. Instructed to maintai...	2026-05-02 14:30:00.0000000
3	3	6	Healthy Devel...	Weight and height are normal for age. Cleared ...	2026-04-20 11:45:00.0000000

	RequestID	AID	Description	DateRequested	Status
1	1	1	Lipid Profile and 12-lead ECG	2026-05-26 18:03:13.4000000	Completed

	PrescriptionID	AID	DrugName	Dosage	Instructions	DatePrescribed
1	1	1	Amlodipine	5mg	Take 1 tablet daily after breakfast	2026-05-26 18:03:13.3966667
2	2	2	Paracetamol	500mg	Take 1 tablet every 4-6 hours as needed for fev...	2026-05-26 18:03:13.3966667

Figure 5.2: Data Population

```
DELETE FROM dbo.LabRequests;
DELETE FROM dbo.Prescriptions;
DELETE FROM dbo.MedicalRecords;
DELETE FROM dbo.Invoices;
DELETE FROM dbo.Appointments;
DELETE FROM dbo.Fee;
DELETE FROM dbo.EmergencyContacts;
DELETE FROM dbo.Doctors;
DELETE FROM dbo.Patients;
DELETE FROM dbo.Users;

DBCC CHECKIDENT ('dbo.Invoices', RESEED, 0);
DBCC CHECKIDENT ('dbo.LabRequests', RESEED, 0);
DBCC CHECKIDENT ('dbo.Prescriptions', RESEED, 0);
DBCC CHECKIDENT ('dbo.MedicalRecords', RESEED, 0);
DBCC CHECKIDENT ('dbo.Appointments', RESEED, 0);
DBCC CHECKIDENT ('dbo.EmergencyContacts', RESEED, 0);
DBCC CHECKIDENT ('dbo.Fee', RESEED, 0);
DBCC CHECKIDENT ('dbo.Patients', RESEED, 0);
DBCC CHECKIDENT ('dbo.Doctors', RESEED, 0);
DBCC CHECKIDENT ('dbo.Users', RESEED, 0);
```

Figure 5.3: Reset and Re-seed Workflow

A maintenance script was written to ensure clean states during iterations:

1. Delete all rows in child-to-parent order, respecting FK constraints.
2. Execute DBCC CHECKIDENT on all tables to reset counters.
3. Re-run the seed INSERT script.

5.2 Code and Interface Implementation

Initially, separating files for each window caused errors; thus, combining them into two files, *main.py* and *data_access_layer.py* (discussed in Chapter 4), fixed those problems. This two-file architecture was the outcome of a deliberate reset in December 2025.

Data Validations

- Required fields validation.
- 11-digit regex for phones; no letters.
- Unique email addresses.
- Strong password.
- Positive consultation fee only.
- Mandatory signature.
- Correct doctor license format.
- Text length limitations.

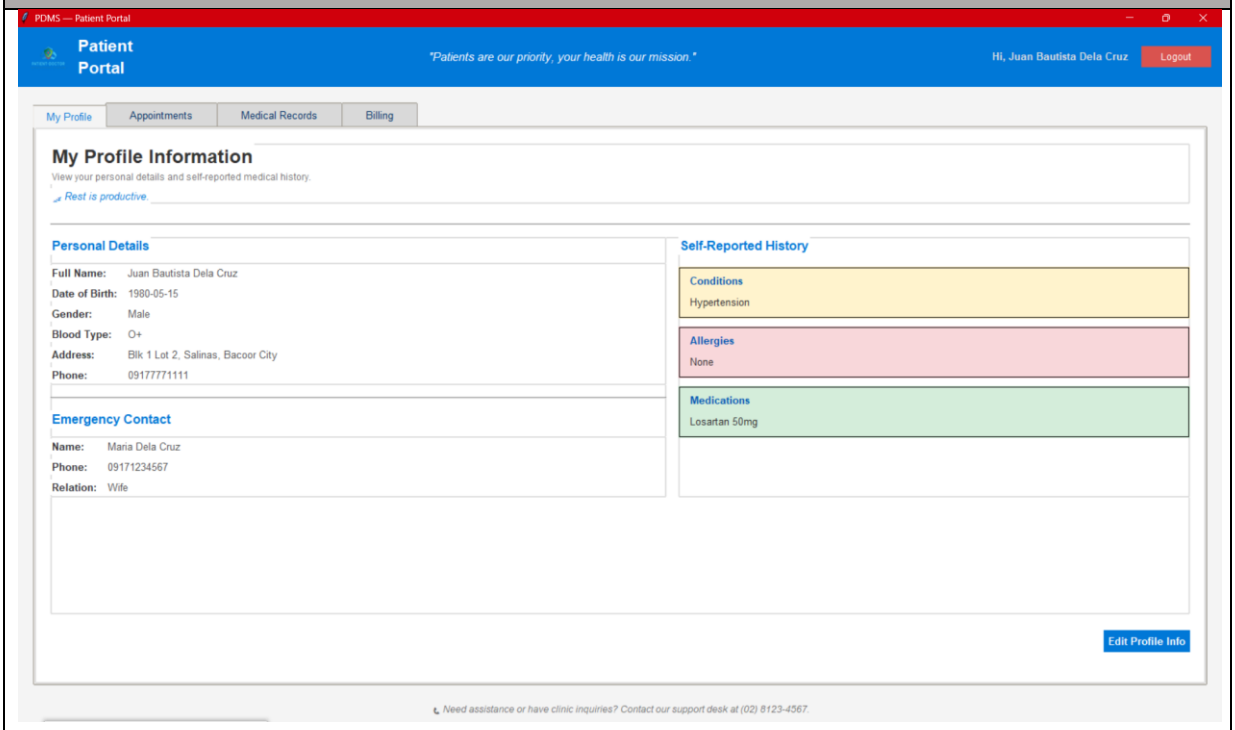
PEP 8 Standards

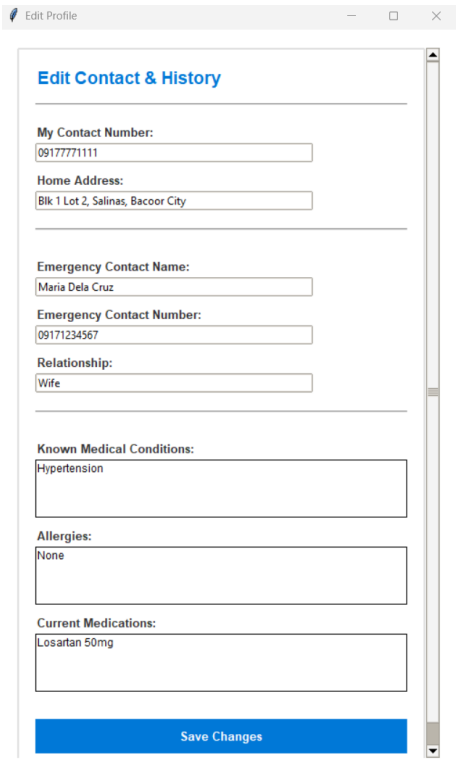
- *PascalCase* for classes.
- *snake_case* for variables and functions.
- Descriptive comments and doctstrings.
- Consistent 4-space indentation
- Separate lines imports.

Aside from OOP, the program follows an event and procedural programming paradigm. The *mainloop()* function maintains an event loop. Both portals integrate *ttk.Notebook* tabbed layout. It consolidates all features into separate tabs within a single window. Furthermore, this approach supports the TAM Perceived Ease of Use principle mentioned in Chapter 2. For consistent visual cues, role-based color schemes (blue for patient, and green for doctor) and a personalized system logo were applied.

Figure 5.4: Profile and Edit Interface

Patient





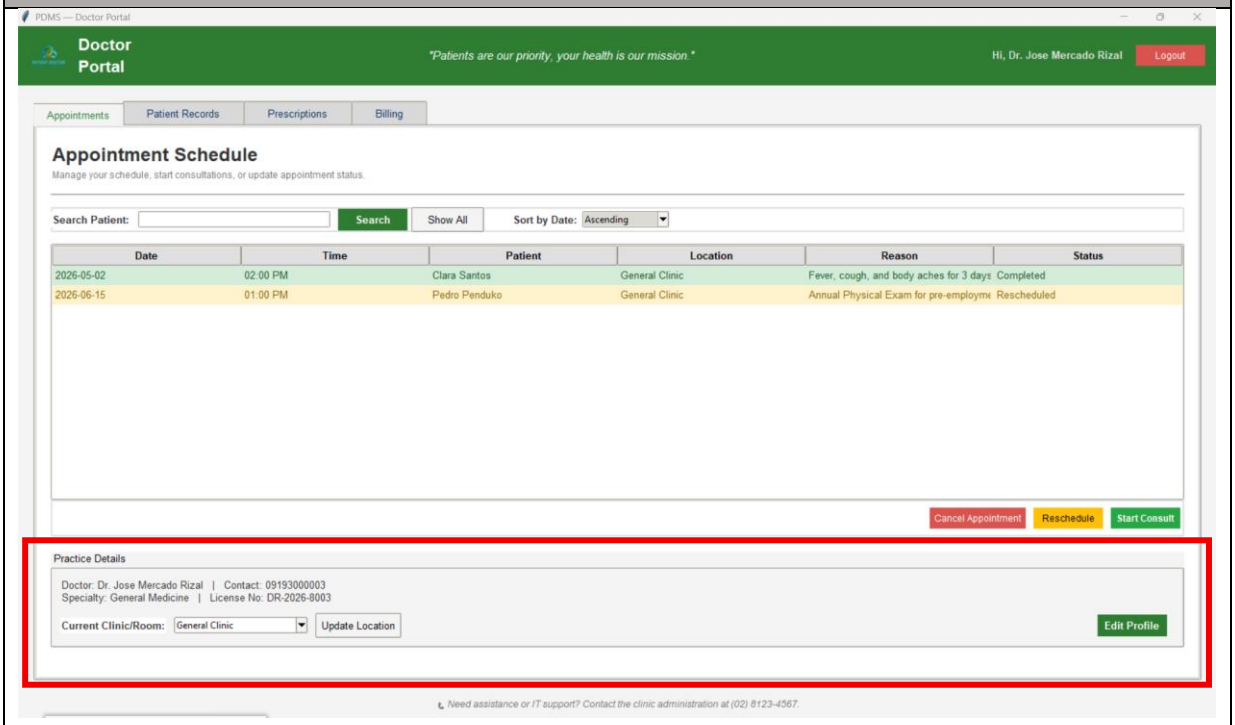
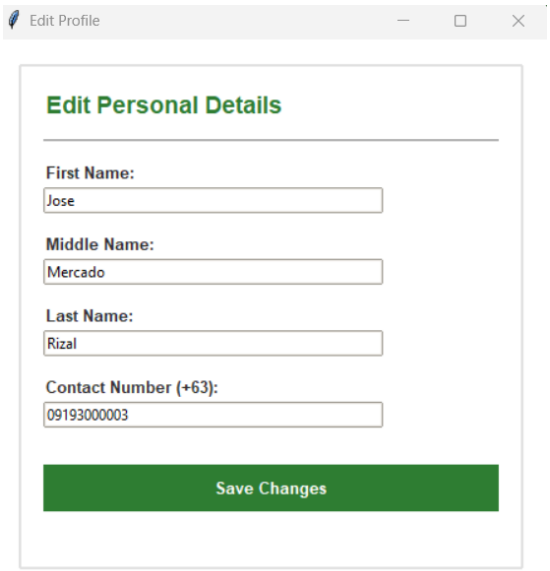
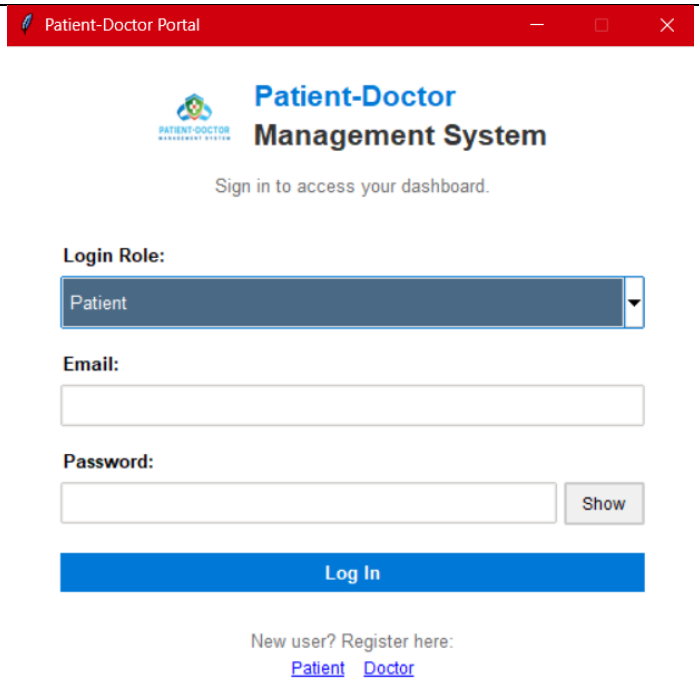
Doctor
   
A *Toplevel* modal opens with the current values. Upon saving, the system validates the fields. It then executes a parametrized UPDATE statement via DAL. On success, *dal.()commit* is called, then refreshes. On failure, *dal.rollback()*. Doctors can update their appointment location, reflected in Treeview and the *Appointments* database.

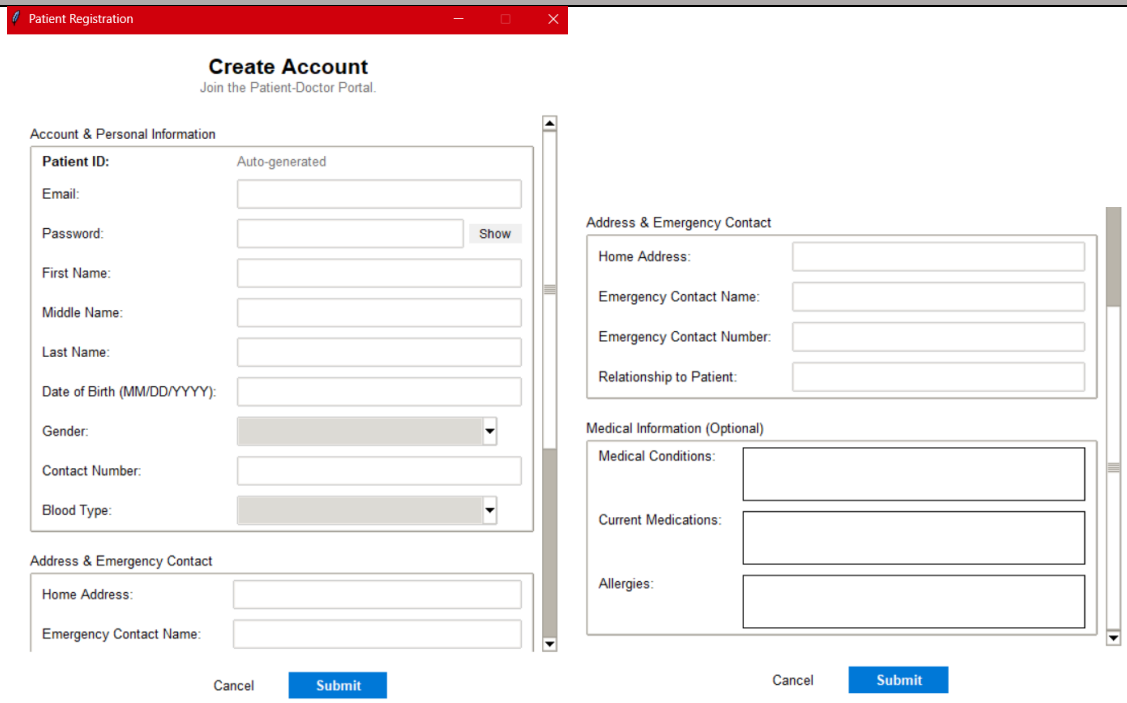
Figure 5.5: Login Interface



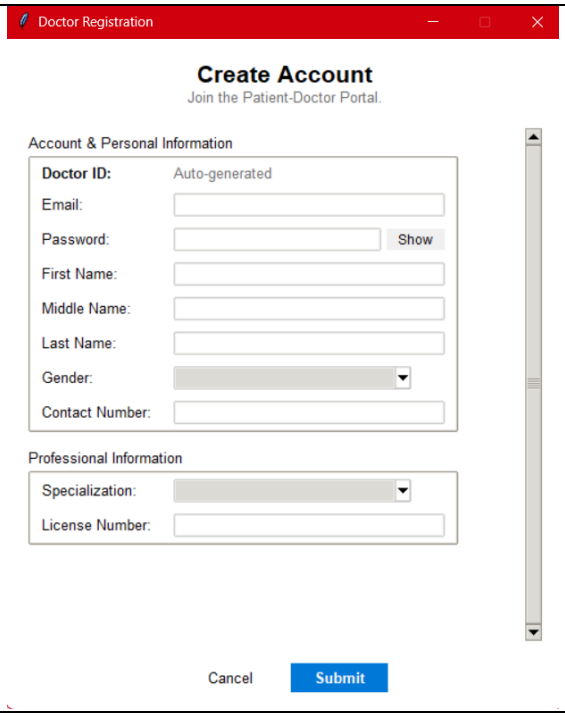
The *LoginWindow* class presents a Combobox to select a role and Entry fields for credentials. The *login_process()* event handler executes a parameterized SQL query against the Users table, which validates *Email*, *PasswordHash*, and *Role* simultaneously. On success, the routing controller *open_main_window()* instantiates either *PatientMainWindow* or *DoctorMainWindow* depending on the returned role, enforcing the Principle of Least Privilege.

Figure 5.6: Registration Interface

Patient



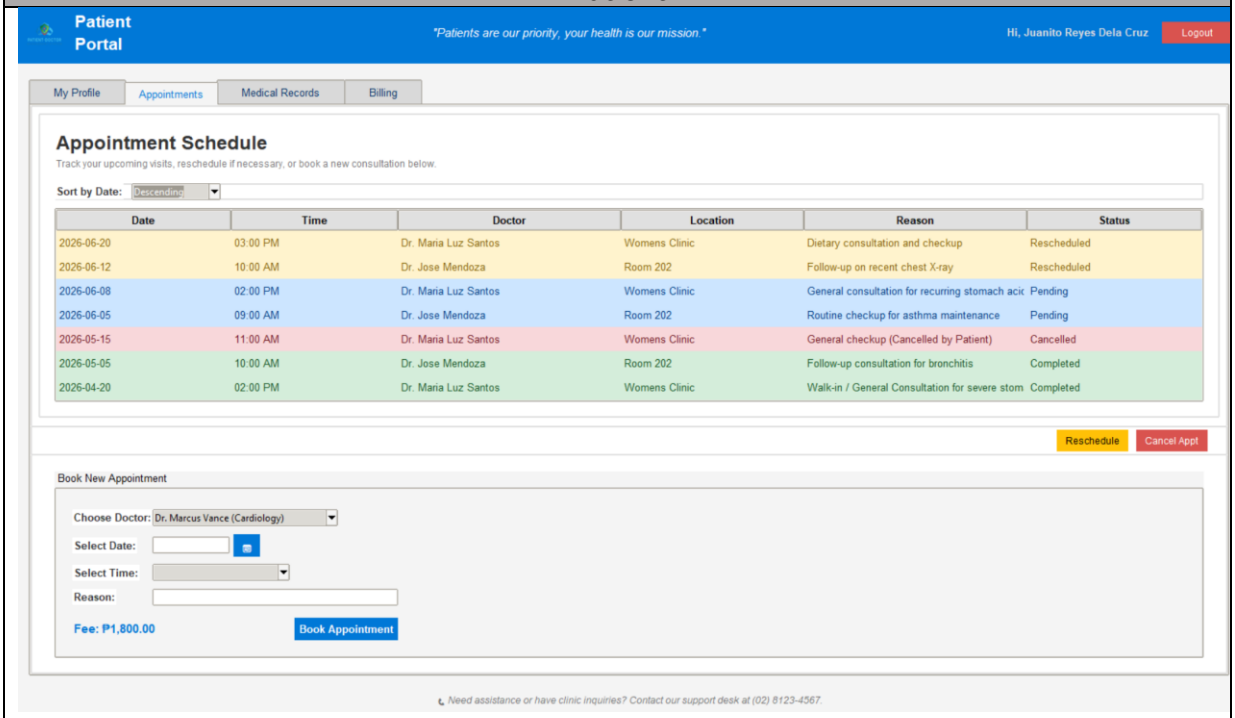
Doctor

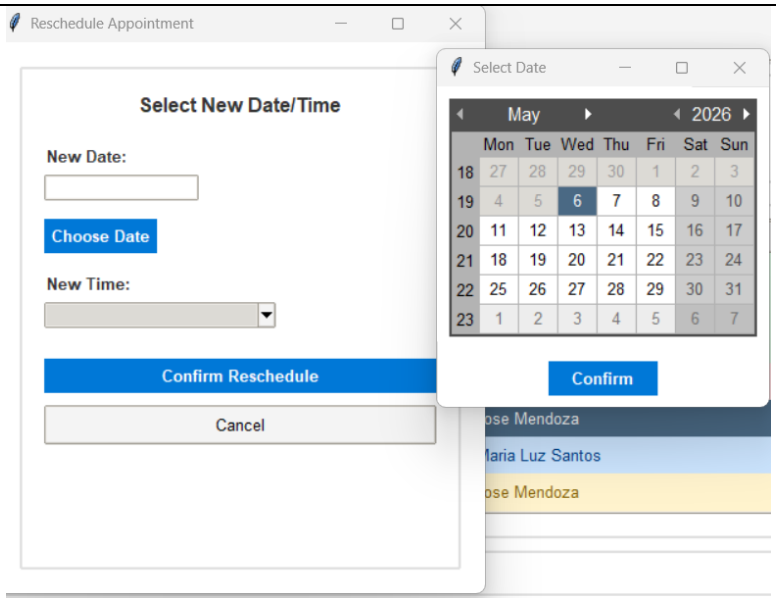


Registration opens when the hyperlink is clicked in the Login Window. *PatientRegistrationWindow* and *DoctorRegistrationWindow* classes utilize Entry and Combobox widgets to obtain user details. Application-layer validation was enforced via regex and.*strip()* methods. Once validated, the data is saved via parametrized INSERT queries using *OUTPUT INSERTED.UserID*, which ensures atomic and cascading writes across the normalized database. Transaction rollbacks support data integrity.

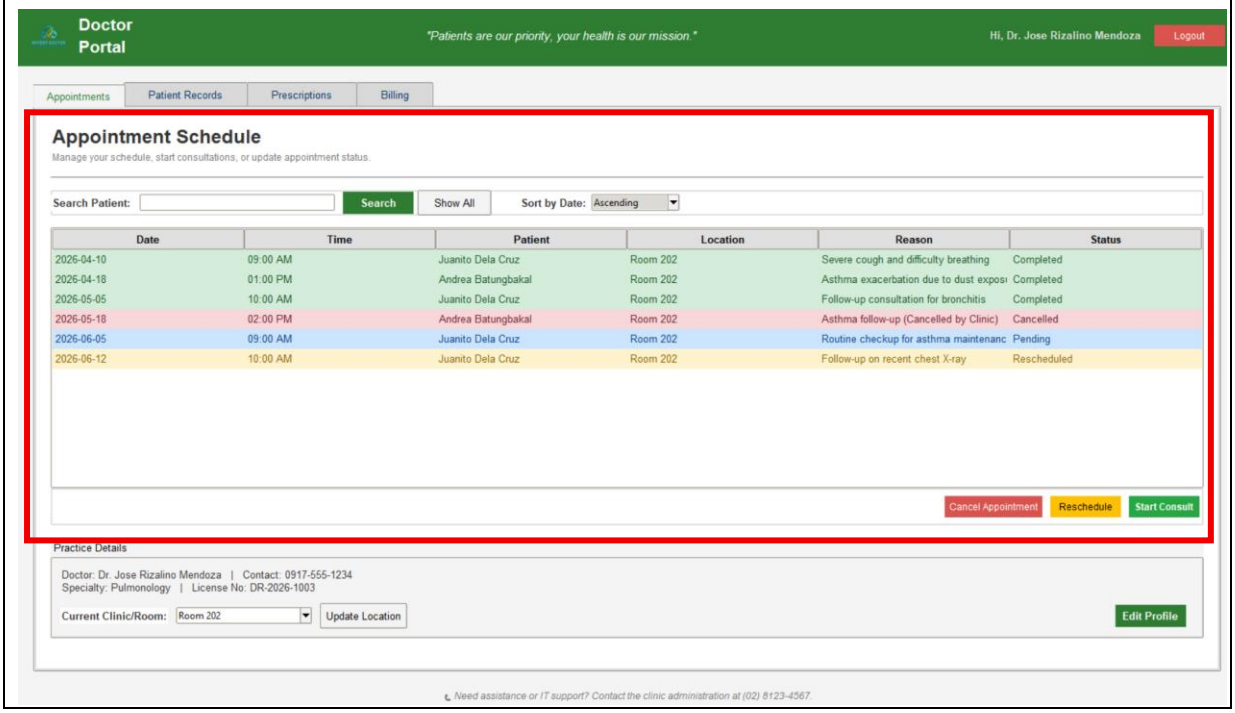
Figure 5.7: Appointments and Reschedule Interface

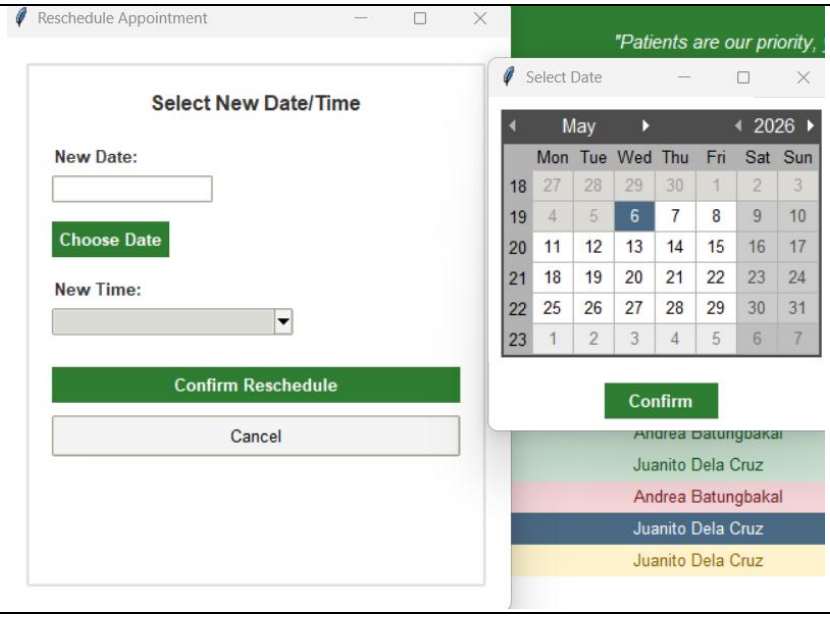
Patient




Reschedule Appointment
Select New Date/Time
New Date:
Choose Date
New Time:
Confirm Reschedule
Cancel
Select Date
May
2026
Mon Tue Wed Thu Fri Sat Sun
Confirm
ose Mendoza
laria Luz Santos
ose Mendoza

Doctor

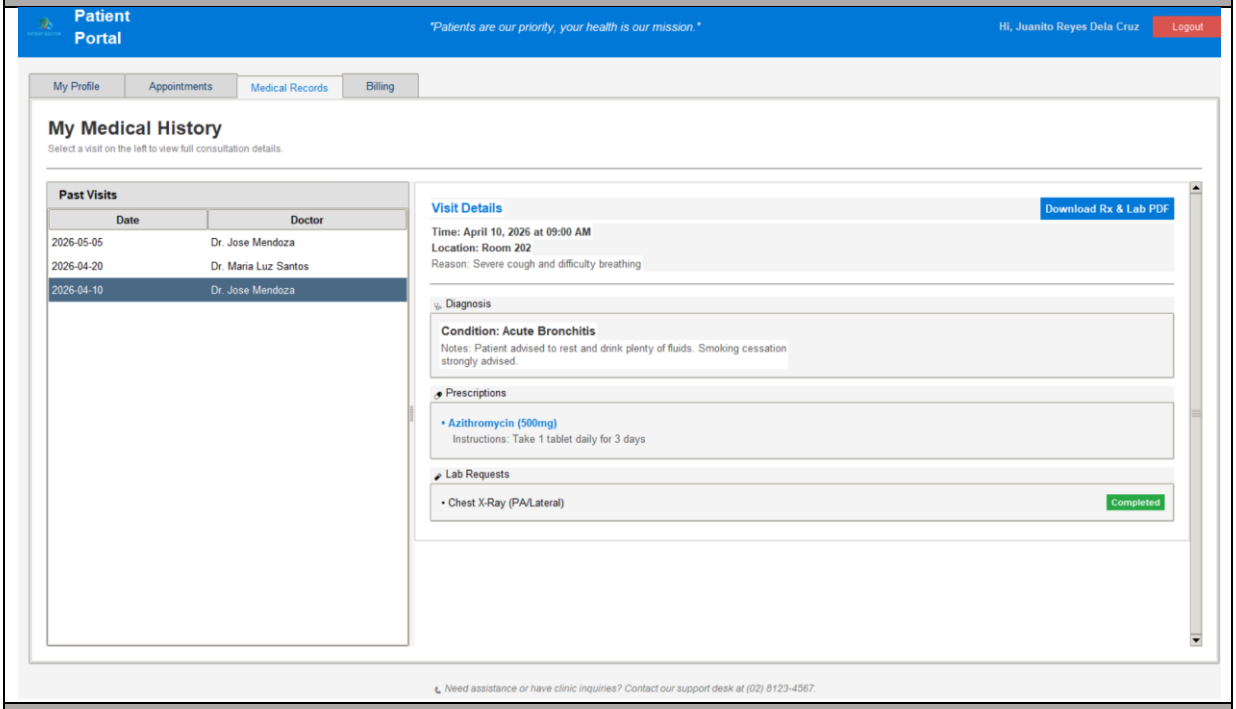

Doctor Portal
"Patients are our priority, your health is our mission."
Hi, Dr. Jose Rizalino Mendoza
Logout
Appointments
Patient Records
Prescriptions
Billing
Appointment Schedule
Manage your schedule, start consultations, or update appointment status.
Search Patient:
Search
Show All
Sort by Date:
Ascending
Date
Time
Patient
Location
Reason
Status
2026-04-10 09:00 AM Juanito Dela Cruz Room 202 Severe cough and difficulty breathing Completed
2026-04-18 01:00 PM Andrea Batungbakal Room 202 Asthma exacerbation due to dust expos Completed
2026-05-05 10:00 AM Juanito Dela Cruz Room 202 Follow-up consultation for bronchitis Completed
2026-05-18 02:00 PM Andrea Batungbakal Room 202 Asthma follow-up (Cancelled by Clinic) Cancelled
2026-06-05 09:00 AM Juanito Dela Cruz Room 202 Routine checkup for asthma maintenanc Pending
2026-06-12 10:00 AM Juanito Dela Cruz Room 202 Follow-up on recent chest X-ray Rescheduled
Cancel Appointment
Reschedule
Start Consult
Practice Details
Doctor: Dr. Jose Rizalino Mendoza | Contact: 0917-555-1234
Specialty: Pulmonology | License No: DR-2026-1003
Current Clinic/Room:
Room 202
Update Location
Edit Profile
Need assistance or IT support? Contact the clinic administration at (02) 8123-4567.



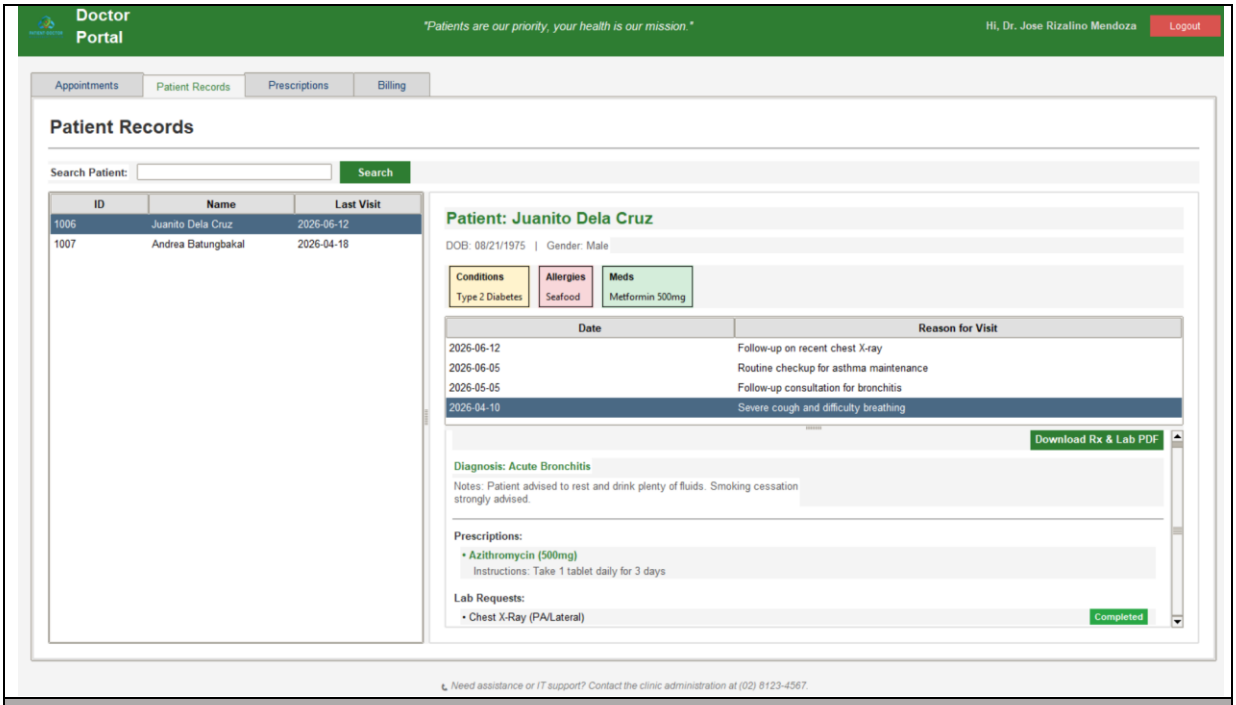
The Appointments tab displays a *ttk.Treeview* of existing, upcoming, and cancelled appointments, and the Patient portal includes a booking form below. The *CalendarPopup* class is widely used for selecting a date. In booking, the *check_timeslots()* procedural function checks all *Pending* and *Rescheduled* appointments, where booked hours are stored in a Python set for membership testing. Clinic hours (08:00-17:00) are marked as 'Available' or 'Booked' and loaded into the Combobox for easier selection. If the patient tries to select a reserved slot, the *ComboboxSelected* event handler blocks the selection.

Figure 5.8: Medical Records Interface

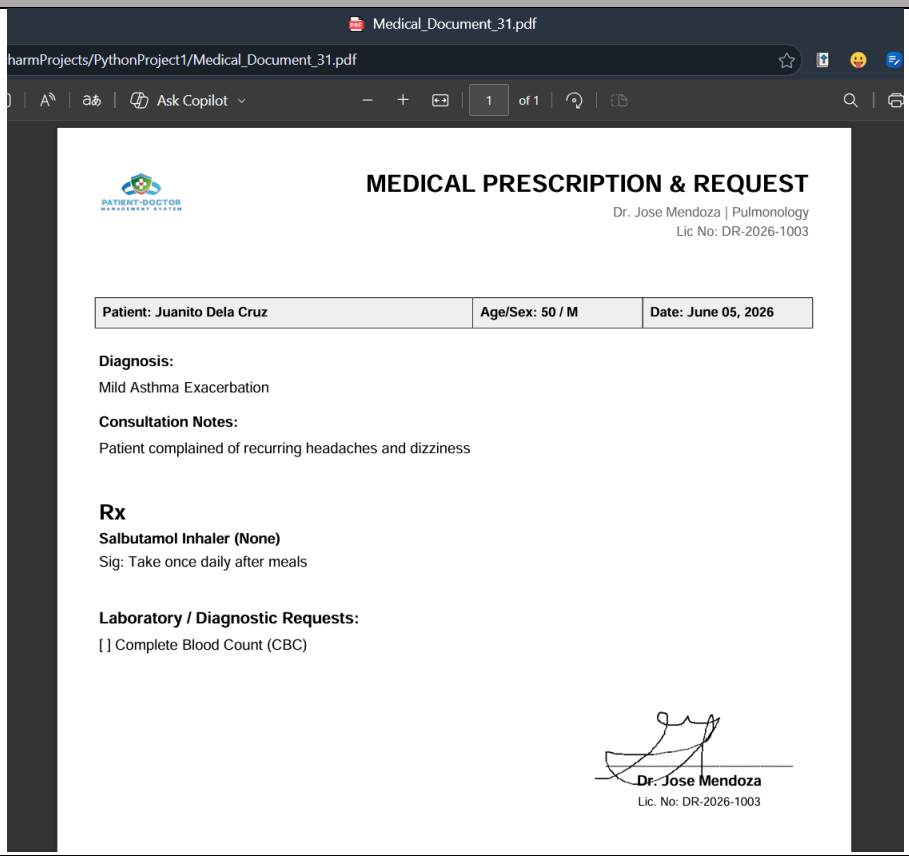
Patient



Doctor



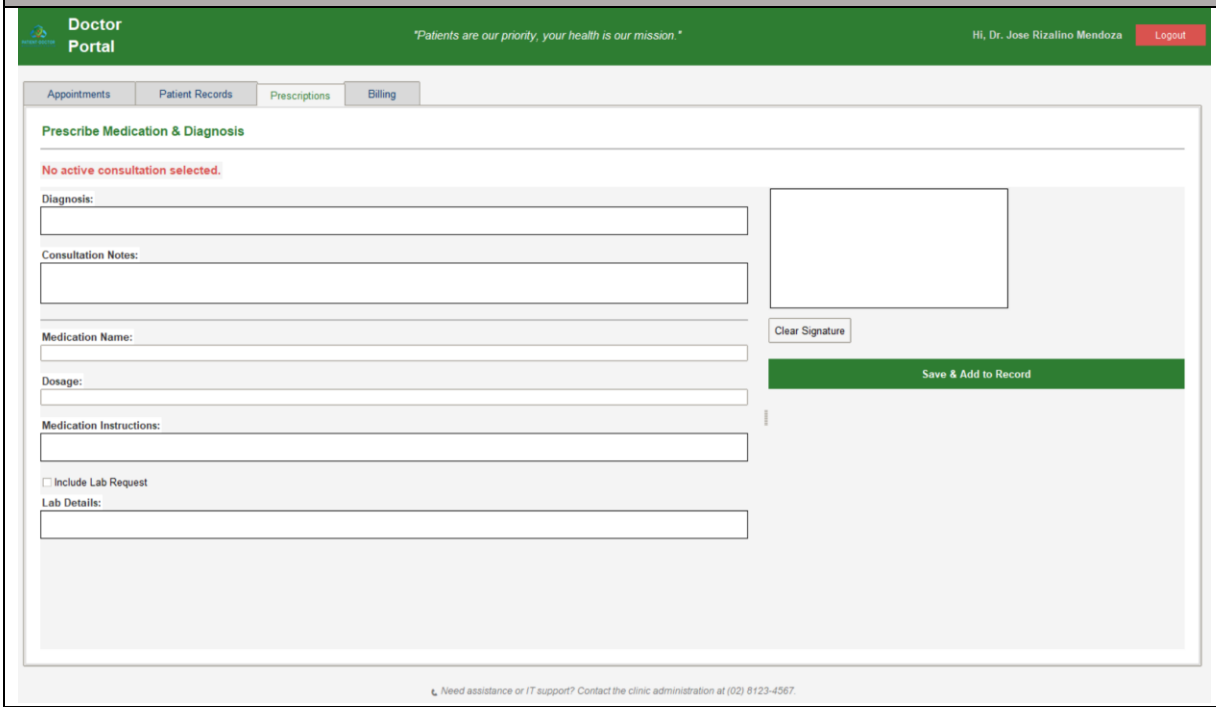
PDF Rx & Lab



The Medical Records tab implements a master-detail layout built with *ttk.PanedWindow*. The left-hand shows the completed appointments (*ttk.Treeview*) with a particular doctor or patient, depending on the perspective. Selecting a row triggers *load_visit()* or *load_consult()*, which sequentially queries the *MedicalRecords*, *Prescriptions*, and *LabRequests* tables for the chosen *AppointmentID*. The *bind_all()* ensures the details are scrollable with the mouse wheel. The doctor portal includes a laboratory status toggle. The Rx PDF was created by initializing FPDF (same approach as Invoice PDF).

Figure 5.9: Prescriptions Interface

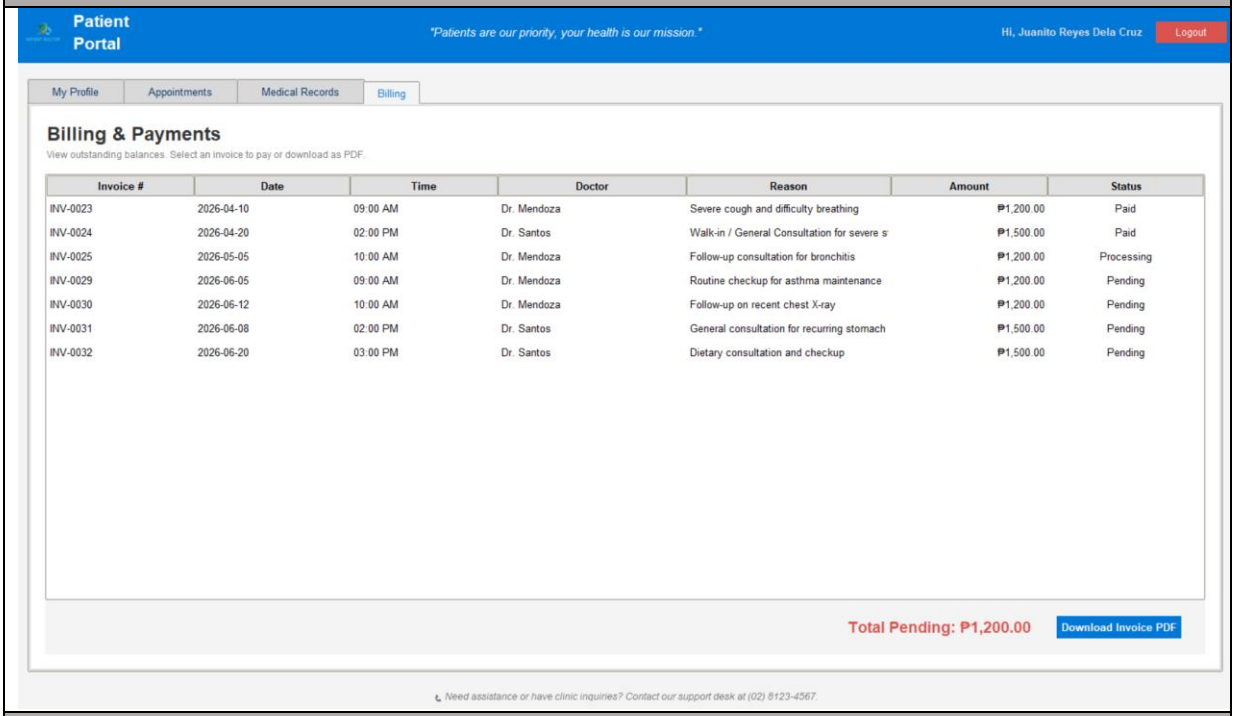
Doctor



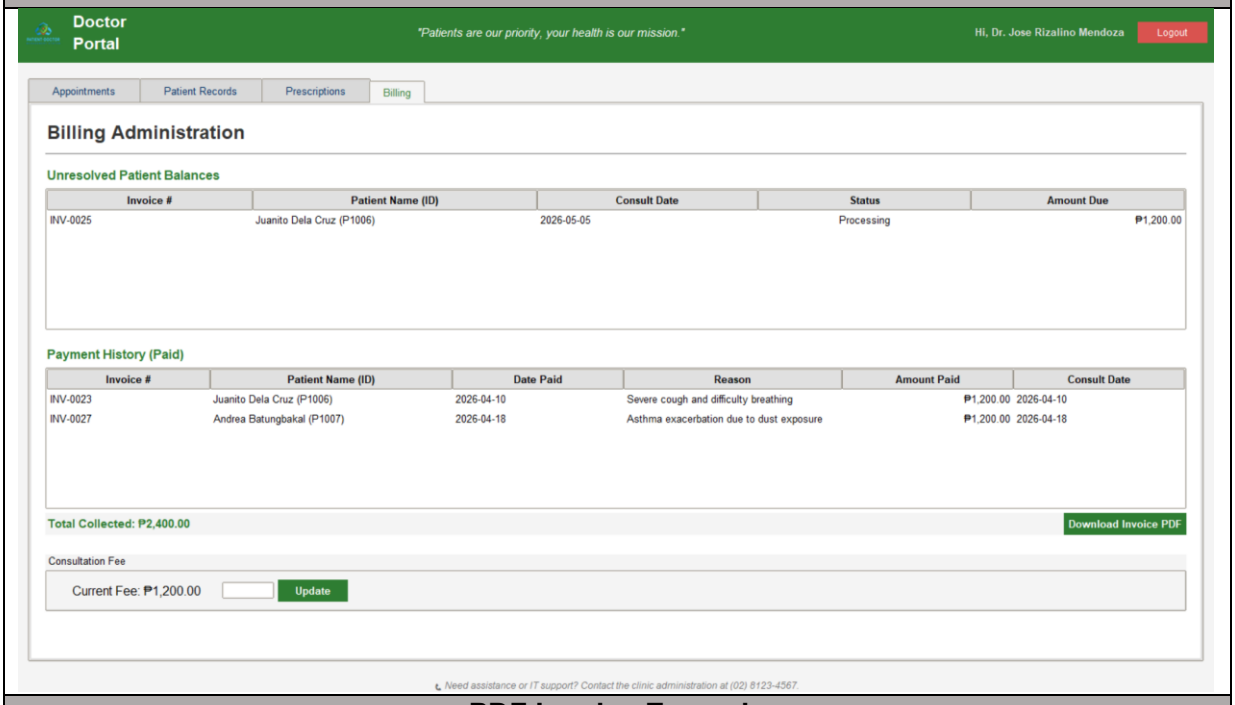
The Prescriptions tab—exclusive to the doctor portal—contains the medication and diagnosis fields with digital signature Canvas on the right. This Canvas is linked to *B1-Motion* events that implement a dual-write strategy, wherein each stroke is rendered in real-time on screen while also being recorded onto a PIL ImageDraw object. Saving the prescription performs an upsert on *MedicalRecords* table *(submit_prescription()),* then atomically transitions both *Appointments.Status* to 'Completed' and *Invoices.Status* to Processing in one DAL transaction. The signature is saved as PNG, keyed to the *AppointmentID*, which is embedded later in the PDF.

Figure 5.10: Billing Management Interface

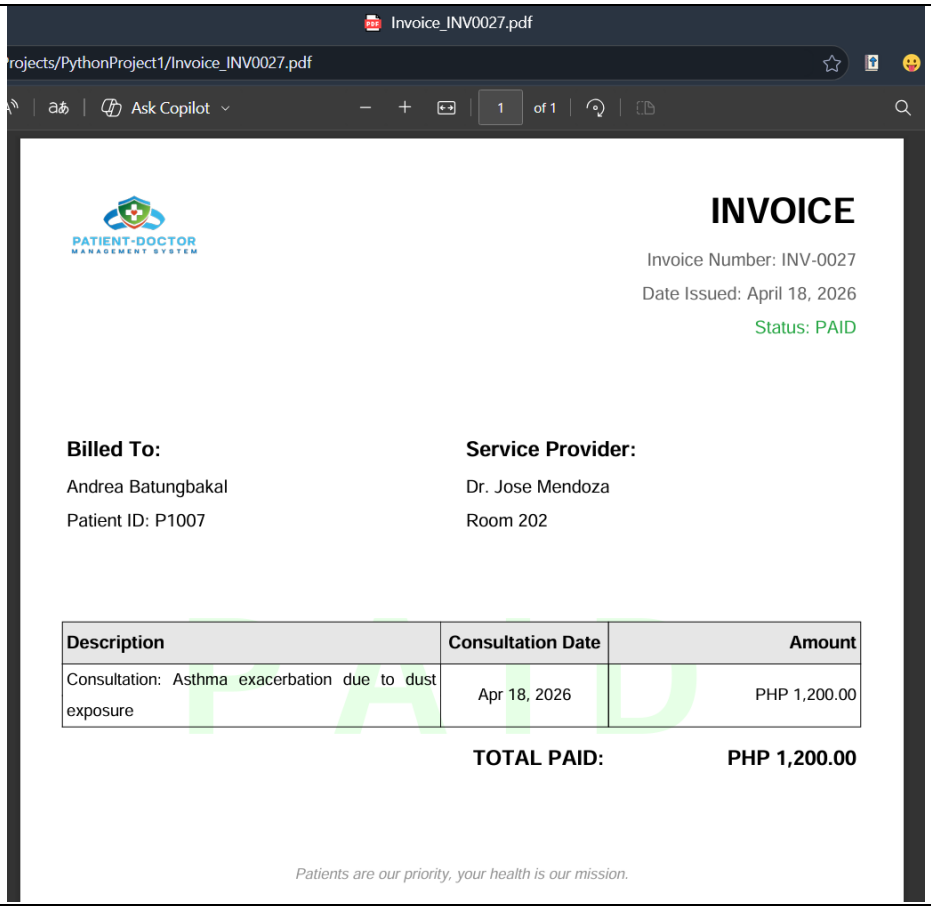
Patient



Doctor



PDF Invoice Example



The Billing tab manages invoices through a three-stage state machine:

1. Pending – created upon booking
2. Processing – auto-escalated by *submit_prescription()*
3. Paid – confirmed when the doctor double-clicks on the unresolved balance in the Treeview, which then triggers *confirm_payment().*

PDF generates utilizes FPDF: *generate_pdf()* performs a multi-table JOIN to aggregate specific clinical data and attach the saved PNG signature, then *download_invoice()* renders a PAID watermark. Python's platform module ensures cross-platform availability by detecting the host OS.

5.3 Challenges and Resolutions

Month	Challenge	Resolution
October 2025	*IDENTITY_INSERT* errors; *PatientID* and *DoctorID* were not independent. Poor database structure.	Refactored to 3rd Normalisation Form. *Patients* and *Doctors* tables reference *Users* via *UID*. All IDs are auto-generated.
	Struggled to recall Python/Tkinter setups and syntax.	Extensive research and independent study through reliable online sources.
November-December	Multiple *Tk()* roots caused window crashes, broken CSS styling, and unresponsive buttons across screens.	Each navigation destroys the current *Tk()* root and creates a new one. Modal windows are now *Toplevel().*

February 2026	Building the registration form caused *_tkinter.TclError* crashes when aligning entry boxes and labels with scrolling containers.	Since *.pack()* and *.grid()* cannot share the same parent frame: *.pack()* – macro-level container frames *.grid()* – internal form fields.
March	SQL truncation errors: doctors accidentally enter long instructions in the *DrugName* field, exceeding the 100-character limit.	Added *len()* validation with error messages.
April	Overlapping text (consultation reasons) in the table of invoice PDFs.	Replaced *cell()* with *multi_cell()* and used *get_x* / *get_y* coordinate tracking to calculate row heights dynamically.
	Appointment table sorted by status instead of date.	Removed CASE grouping; added ASC/DESC filter.
	Patients could download unpaid invoices.	Added a status check in *download_invoice()* so only Paid Invoices are downloadable.
May	Doctors could submit prescriptions without their signature.	Added canvas validation using *sig_canvas.find_all()*; blocks submission of empty signature.

To conclude, the developed prototype aligns with the modern clinical administrative requirements and software engineering principles. Adaptability, critical thinking, and problem-solving skills produced a more robust and maintainable system, which will be further confirmed through the testing methodologies in Chapter 6.

Chapter 6

This chapter presents functional, performance, and boundary testing methods. The results are then analyzed through the ISSM and TAM frameworks, mentioned in Chapter 3, to evaluate whether the system is hypothetically accepted, reliable, and useful in rural settings. See *Appendix C* for proof of error interception.

6.1 Black-Box Testing

Black-box testing is evaluating the system's functionality through user inputs, outputs, and expected behavior, without inspecting internal code (Das, 2024). This method makes it user-centric, considering it is intended for non-technical clinic users.

			Outcome		
Test ID	**Module**	**Test Case**	**Expected**	**Actual**	**Status**
TC-01	**Authentication**	Valid patient login.	Portal opens, doctor is blocked.	Achieved	**Pass**
TC-02		Valid doctor login.	Portal opens, patient is blocked.		
TC-03		Incorrect password.	Error message shown.		
TC-04		Role mismatch.			
TC-05	**Registration**	Valid user form.	Account created; return to Login.		
TC-06		Invalid doctor license.	Error message shown.		
TC-07		Duplicate email.			
TC-08		Weak password.			
TC-09	**Appointments**	Book a valid slot.	Appointment and 'Pending' invoice created.		
TC-10		Book an occupied slot.	Pop-up message shown.		
TC-11		Reschedule a completed appointment.			
TC-12		Reschedule a scheduled appointment.	Record updated.		
TC-13	**Prescriptions**	Valid prescription.	Record saved.		
TC-14		Lab request only.			
TC-15		Drug name is >100 characters.	Error message shown.		
TC-16		No signature.			
TC-17	**Medical Records**	View medical record/s.	Data displayed.		
TC-18		Doctor toggle laboratory status.	Status updated.		
TC-19		Download Rx.	PDF generated.		
TC-20	**Billing**	Download unpaid invoice.	Pop-up message shown.		

TC-21		Pending -> Processing -> Paid	Sequential		
TC-22		Download paid invoice.	PDF generated.		
TC-23		Update consultation fee.	Data updated.		
TC-24	**Edit Profile**	Patient update.	Data updated and refreshed.		
TC-25		Doctor update.			

6.2 Boundary Testing

Boundary Value Analysis (BVA) intercepts extreme and invalid inputs before reaching the database layer (Dssouli, 2017). This is to ensure 100% data accuracy and integrity.

Data Field	**Boundary Constraint**	**Tested Entries**	**Result**
Password	Min: 12 chars	Pass@word12 Pass@word123	11 blocked; 12 accepted
Appointment Date	Min: date.today()	2026-05-11 (Yesterday), 2026-05-12 (Today)	Yesterday, greyed out; Today accepted
Consultation Fee	Min: 0.00	-1.00, 0.00	Negative blocked; Zero accepted
License	Format: XX-YYYY-XXXX	DR-25-0001 (Invalid) DR-2025-0001 (Valid)	Invalid format blocked; Valid accepted
Contact Number	Max: 20	20, 21	20 saved; 21 blocked
Patient/Doctor Name, Dosage	Max: 50	50, 51	50 saved; 51 blocked
Drug Name	Max: 100	100, 101	100 saved; 101 blocked
Instructions, Address	Max: 255	255, 256	255 saved; 256 blocked
Diagnosis	Max: 500	500, 501	500 saved; 501 blocked
Medical Notes, History	Free-text	Standard paragraph	Accepted

6.3 Performance Testing

This test will be performed on Windows 11, a local SQL Service instance, and a seed dataset of five doctors and patients to verify the project's measurable objectives; results reflect a prototype environment.

Obj. #	**Objective**	**Target**	**Test Performed**	**Measured Result**	**Verdict**
O1	100% offline operation	Full workflow	Disabled Wi-Fi connection; executed login → book → prescribe → invoice	Operated normally, zero connectivity.	**Pass**
O2	Patient data retrieval speed	<3s	Navigating a patient record, timed until data is populated.	Average 1.7s across 3 trials.	
O4	Doctor consultation efficiency	<5 mins., <12 clicks	Timed consult flow; counted clicks.	Average 2m 47s; 8 clicks to complete all fields.	

O5	Data accuracy and response time	100%, <2.5s	1) Verified BVA logs 2) Timed tab switch.	1) 100% malformed data blocked 2) 1.3s medical records, 1.4s billing.	
O6	Appointment booking speed	<5 mins.	Timed full booking flow.	Avg. 1m 53s across 3 trials.	

6.4 Heuristic Evaluation

Nielsen's Ten Usability Heuristics evaluates the quality of GUI design against established principles to determine issues in early development (Kalsha, 2025). This assessment was done as a preliminary self-assessment by the developer; a formal review by UX practitioners is recommended in future work.

Heuristic	Observation	Status
Visibility	• Invoice and appointment statuses are indicated. • Header displays the user's name.	**Full Functionality**
Match	Clinical terminologies, billing, and appointment scheduling processes reflect the actual clinic setting.	
Consistency	Portals utilize unified color schemes and layouts.	
Error Prevention	• Greyed out booked slots. • Data validation fields. • Terminal states block edits.	
Recognition	• Dropdowns simplify input. • The table view/s displays the necessary information.	
Flexibility	"Start Consult" shortcut lessens navigation steps.	
Aesthetic	Tabs have their own designated task.	
Error Recovery	Designated error dialogs identify failed fields.	
User Control	• Cancel and reschedule appointments. • Overwrite data (e.g., prescriptions, fee). However, no undo actions.	**Partial Functionality**
Help	No tooltips or in-app guides, only a research paper and video demonstration.	

6.5 Predictive Assessment (ISSM/TAM)

Since full clinical deployment is outside of the project's scope, empirical survey data were not collected. Instead, this section provides a predictive analysis of how the system conceptually aligns with the aforementioned frameworks based on the implemented features.

Information Systems Success Model		
Dimension	**Evidence**	**Assessment**
Information Quality	• Normalized tables contain clinical data. • Accurate invoices. • PDFs consolidate records.	High – traceable, accurate, consistent outputs.

	• Database constraints for consistency.	
System Quality	• Three-tier architecture • Data Access Object • Data validations • Atomic transactions • Offline functionality	High – stable on normal use, resilient to edge cases.
Service Quality	• UX as proxy. • Shortcuts and role portals reduce friction. • No IT desk.	Moderate – effective workflows; absence of help documentation
Intention to Use/User Satisfaction	• 8/10 heuristics satisfied. • Real-world vocabulary supports usability.	Neutral – structured user testing needed.
Net Benefits	• Replaces paper workflows. • Integrates medical records, appointments, and billing.	Predictive benefits, pilot validation required.
Technology Acceptance Model		
Perceived Ease of Use	• Tabbed layout. • Straightforward buttons.	Usable, but the onboarding gap remains.
Perceived Usefulness	• Secure sensitive data. • Consolidate records offline.	Useful utility; lack of reporting/analytics.
Intention to Use -> Actual Use	• Two dimensions are positive. • Offline for low-connectivity clinics.	Favorable, supports adoption.

All 25 black-box cases passed. All 5 quantitative objectives met their specified threshold. Boundary testing confirms the graceful handling of the unusual values, and heuristic evaluation satisfied 8 of 10 principles. For frameworks, ISSM indicated strong information and system quality, but moderate service quality, while TAM showed high usefulness and ease. Chapter 7 reflects on the entire development and results of PDMS.

Chapter 7

This chapter reviews the Patient-Doctor Management System against the eight objectives defined in Chapter 1, drawing from the test results in Chapter 6. It examines the extent to which each objective was accomplished, identifies the system's strengths and limitations, and outlines strategic recommendations for future development.

7.1 Analysis of Objectives

Objectives 1 and 3 were fully achieved. Internet dependency was eliminated by following a three-tier architecture, Python Tkinter, and a local SQL Server deployment. All database tables are managed through *pyodbc* with atomic transactions through DAL. RBAC secures and isolates the Patient and Doctor portals effectively (Patient cannot access the Doctor Portal, and vice versa). This made the researcher feel confident in the ability to design a self-contained and resilient system. Furthermore, objectives 7 and 8 were fully achieved by applying SDLC phases and 3NF normalization to the database, and going through it was a steep learning curve. The researcher attempted to build an EHR-like prototype, which led to extensive hands-on experience in healthcare informatics and reinforced knowledge on clinical data, privacy, and workflows. This prototype will help me create a relevant portfolio that aligns with the researcher's career interests in healthcare.

Objectives 2, 4, 5, and 6 are predictively achieved and highlight strict operational benchmarks. In Chapter 6, the system theoretically exceeded the targets. While the GUI was optimized to minimize frictions, these temporal metrics remain dependent on clinical variables and human typing speed. Drawing from the boundary value testing, Chapter 5 was achieved through rigorous defensive programming (multi-layer validation). The system successfully intercepts invalid inputs through SQL Server constraints and formatting regular expressions; however, the correctness of free-text fields like diagnosis notes is user-dependent and remains an absolute limitation.

These metrics taught the researcher the impact of indexing PKs and normalization on speed. Since testing relied on simulated data and was conducted on a personal computer, the objectives are considered predictively achieved, which makes it a sufficient baseline for future field validation. To validate this empirically in a real deployment, a formal study with a real clinic staff is recommended.

7.2 System Strengths and Limitations

The PDMS successfully delivered a fully functional clinic management prototype. A notable strength of PDMS is the depth of defensive programming: multi-layer validation, role-isolated architecture, and atomic transactions, all achieved within five months of solo development. Compared to the initial wireframes, the final interfaces are significantly more logical and cleaner with their visual cues. Some optional features marked as MoSCoW prioritized were successfully delivered, exceeding the originally planned scope.

However, bounded by time, knowledge, and resource constraints, some limitations were identified. First, disaster recovery is absent, like remote backup or cloud-sync, making the data vulnerable to hardware failure. Second, concurrency was untested; simultaneous write requests from multiple clinic staff may cause database lock conflicts. Third, the system lacks a dedicated Medical Assistant or Administrator portal. Fourth, cash-only payments are assumed; there are no online payments. Finally, in emergencies, physical paper remains faster and more

accessible, wherein users may prefer writing over typing. This is potentially a usability constraint that can be addressed in future iterations.

7.3 Recommendations

Since this is just a prototype, it can be expanded into a usable product through future iterations upon its existing architecture. This includes:

1. Integrating digital payment gateways (e.g., GCash, online banking) and an HMO (Health Maintenance Organisation) insurance can modernize and provide variety in patient billing.
2. Developing a third RBAC tier for medical assistants or administrators to manage administrative tasks.
3. Implementing a clinic dashboard containing data visualization that can provide insights into patient volume and medical trends.
4. Integrating Artificial Intelligence to analyze medical histories and predict patient risks.
5. Further normalising of the schema to support one-to-many relationships for multi-illness diagnoses and dynamic clinic locations.

The Patient-Doctor Management System successfully demonstrated the feasibility of a normalized, offline healthcare tool designed for small, rural Philippines clinics. Software budgets and internet connectivity remain barriers for these clinics; thus, this project offers a practical stepping stone toward a digital transformation. Defensive programming and strict UI/UX design ensured a resilient system. Moreover, the development process highlighted that the user interface is only a fraction of the system; the majority lies in logic and intention to protect data integrity. Above technical achievement, this journey taught the researcher that developing healthcare systems requires a balance of reliability with usability and anticipating real-world constraints, compliance, and human needs. Ultimately, the PDMS achieved its core objectives, which provide a sufficient foundational prototype for future refinements.

References

Acacio-Claro, P.J. et al. (2024) Understanding Adoption of Electronic Medical Records (EMRs) during a Health Emergency: An Analysis of EMR Usage Logs from Rural Health Facilities in the Philippines. Asian Medical Professionals, [Online] Paper presentation – International Teleconference on Technology and Policy for Supporting Implementation of COVID-19 Recovery Plan in Southeast Asia (ITTP-COVID19), August 6–8, 2021 (online). Available from: https://doi.org/10.47895/amp.vi0.8911 [Accessed 18/03/2026].

Amor C. Tinam-isan, M. and F. Naga, J. (2024) Exploring the Landscape of Health Information Systems in the Philippines: A Methodical Analysis of Features and Challenges. International Journal of Computing and Digital Systems, 15(1), pp. 225–237.

Berezhna, M. (2025) Separation of Concerns: A Fundamental Principle in Software Development. [Online] DEV Community. Available from : https://dev.to/marie_berezhna/separation-of-concerns-a-fundamental-principle-in-software-development-oaj [Accessed 05/05/26].

Boosted CRM. (2025) [Online] BoostedCRM. Available from : https://boostedcrm.com/clinic-management-system-features/ [Accessed 02/05/26].

Cao, W., Wan, Y., Tu, H. et al. (2011) 'A web-based appointment system to reduce waiting for outpatients: A retrospective study', BMC Health Services Research, 11, p. 318. doi: 10.1186/1472-6963-11-318

Das, S. (2024) What is Black Box Testing: Types, Tools & Examples | BrowserStack. [Online] BrowserStack. Available from : https://www.browserstack.com/guide/black-box-testing [Accessed 12/05/26].

Davis, F.D. (1989) Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Quarterly, 13(3), pp. 319–340.

de Carvalho Junior, M. A., & Bandiera-Paiva, P. (2018). Health Information System Role-Based Access Control Current Security Trends and Challenges. Journal of healthcare engineering, 2018, 6510249. https://doi.org/10.1155/2018/6510249

De Mesa, R. Y. H., Galingana, C. L. T., Tan-Lim, C. S. C., Javelosa, M. A. U., Panganiban, J. M. S., Fabian, N. M. C., Calderon, Y., Rey, M. P., Bernal-Sundiang, N., Sanchez, J. T., Dans, L. F., Casile, R. U., & Dans, A. L. (2024). Facing the digital frontier: Exploring user acceptance of electronic health records in an urban, rural and remote setting in the Philippines. PMCID: PMC11029422.

Delone, W. and McLean, E. (2003) The DeLone and McLean Model of Information Systems Success: a Ten-Year Update. Information Systems Research, Journal of Management Information Systems, 19(4), pp. 9–30.

Dssouli, R. et al. (2017) Testing the Control-Flow, Data-Flow, and Time Aspects of Communication Systems: A Survey. Advances in Computers, pp. 95–155.

Ebardo, R.A. and Celis, N.J. (2019) Barriers to the Adoption of Electronic Medical Records in Select Philippine Hospitals. Proceedings of the 2019 5th International Conference on Computing and Artificial Intelligence, pp. 123–128.

Elisasen, B., Pedersen, C. D., & Szpisjak, A. (2025). The Nordic way: Denmark's EHR approach and regional integration. In Digital Maturity in Hospitals (pp. 151–167).

Eswar, M. S. (2025). Cloud-based patient record management system for rural clinics: A scalable and offline capable solution. International Research Journal of Engineering and Technology, 12(4), 371–372.

GeeksforGeeks (2017) Data Access Object(DAO) Design Pattern. [Online] GeeksforGeeks. Available from : https://www.geeksforgeeks.org/system-design/data-access-object-pattern/ [Accessed 05/05/26].

GeeksforGeeks (2019) Evolutionary Model Software Engineering. [Online] GeeksforGeeks. Available from : https://www.geeksforgeeks.org/software-engineering/software-engineering-evolutionary-model/ [Accessed 29/04/26].

Hoory, L. (2022) What Is An EMR? Everything You Need To Know. [Online] Forbes Advisor. Available from : https://www.forbes.com/advisor/business/software/what-is-an-emr/ [Accessed 28/04/26]. https://doi.org/10.2196/medinform.8131

Kinza Yasar and Gillis, A.S. (2025) What is the principle of least privilege (POLP)? [Online] Search Security. Available from : https://www.techtarget.com/searchsecurity/definition/principle-of-least-privilege-POLP [Accessed 05/05/26].

Leyso, N.L. and Umezaki, M. (2024) Spatial inequality in the accessibility of healthcare services in the Philippines. GeoJournal, 89(3), [Online] Available from: doi.org/https://doi.org/10.1007/s10708-024-11098-3 [Accessed 18/03/2026].

Macabasag, R.L.A. et al. (2022) Normalisation of electronic medical records in routine healthcare work amidst ongoing digitalisation of the Philippine health system. Social Science & Medicine, 307, p. 115182.

Malanin, V. (2025). Implementing offline-first web apps for remote healthcare monitoring. International Journal of Research Publication and Reviews, 6(5), 12351–12361.

Moran, K. and Gordon, K. (2023) How to Conduct a Heuristic Evaluation. [Online] Nielsen Norman Group. Available from : https://www.nngroup.com/articles/how-to-conduct-a-heuristic-evaluation/ [Accessed 29/04/26].

Musunuri, S. E. (2025). Cloud-Based Patient Record Management System for Rural Clinics: A Scalable and Offline Capable Solution. International Research Journal of Engineering and Technology (IRJET), 12(4), 371-372.

Ngafeeson, M. (2014) The Commons Healthcare Information Systems: Opportunities and Challenges.

Nur Fi'llia Nugrahani (2025) Best 10 Clinic Software for Streamlined Healthcare in the Philippines. [Online] HashMicro Philippine Blog. Available from : https://www.hashmicro.com/ph/blog/clinic-software/ [Accessed 02/05/26].

Ongkeko, Jr., A.M. et al. (2016) Community Health Information and Tracking System (CHITS): Lessons from Eight Years Implementation of a Pioneer Electronic Medical Record System in the Philippines. Acta Medica Philippina, 50(4), [Online] Available from: doi.org/https://doi.org/10.47895/amp.v50i4.769 [Accessed 18/03/2026].

Schorr, A. (2023) The Technology Acceptance Model (TAM) and Its Importance for Digitalization Research: a Review. Sciendo eBooks, pp. 55–65.

Shashikant Kalsha (2025) Heuristic Evaluation Guide: Step-by-Step Process. [Online] QodeQuay | Human-Centered Design Thinking & Technology Consulting. Available from : https://www.qodequay.com/how-to-conduct-a-heuristic-evaluation [Accessed 13/05/26].

Smith, P. (2025) What is EMR in Medical Billing? Its Benefits, Uses, and Key Role. [Online] HMS USA inc. Available from : https://hmsgroupinc.com/what-is-emr-in-medical-billing/ [Accessed 28/04/26].

Syzdykova, A., Malta, A., Zolfo, M., Diro, E., & Oliveira, J. L. (2017). Open-Source Electronic Health Record Systems for Low-Resource Settings: Systematic Review. JMIR medical informatics, 5(4), e44.

Urbach, N. and Müller, B. (2011) The Updated DeLone and McLean Model of Information Systems Success. Integrated Series in Information Systems, pp. 1–18.

Yi, S., Yam, E. L. Y., Cheruvettolil, K., Linos, E., Gupta, A., Palaniappan, L., Rajeshuni, N., Vaska, K. G., Schulman, K., & Eggleston, K. N. (2024). Perspectives of digital health innovations in low- and middle-income health care systems from South and Southeast Asia. Journal of Medical Internet Research, 26, e57612.

Appendices

Word Count Breakdown

Word count requirement: 6500 (+10%) = 7150 words maximum

- Abstract – 113
- Chapter 1 – 871
- Chapter 2 – 1,009
- Chapter 3 – 996
- Chapter 4 – 1,167
- Chapter 5 – 1,093
- Chapter 6 – 959
- Chapter 7 – 745

  Exclude Appendices

Total = 6,953 words

Appendix A: Use Case Descriptions

User Access Subsystem

Use case number: UC-01	**User case name:** Register Account
Brief description	The user creates either a Patient or a Doctor account by providing personal and credential information.
Actors	Patient, Doctor
Preconditions	The user has launched the PDMS application and is on the Login Window.
Postconditions	A new Users row and its corresponding Patients or Doctors are inserted in the database. The user is automatically returned to the Login Window.
Primary path	1. User clicks either the ‘Patient’ or ‘Doctor’ link on the Login Window. 2. User fills all the fields. 3. User clicks the ‘Submit’ button.
Alternatives	User clicks ‘Cancel’. This closes the registration window and restores the Login Window.
Exceptions	Duplicate email detected, invalid field format (e.g., weak password, wrong license format).
Data Requirements	Email, password, role, first/middle/last name, date of birth, gender, and contact number. Patient: blood type, home address, emergency contact, and optional medical background. Doctor: specialization, license number.
User Interfaces	Figure 5.6: Registration Interface

Use case number: UC-02	**User case name:** Log-in to Account
Brief description	An existing user authenticates by providing their role, email, and password to access the portal.
Actors	Patient, Doctor
Preconditions	The user has a registered account. The login window is open.
Postconditions	Patient Portal or Doctor Portal is loaded. The Login Window is destroyed.
Primary path	1. User selects role and enters their email and password. 2. User clicks ‘Log In’.
Alternatives	None.
Exceptions	No matching record. Empty fields.
Data Requirements	Email, password, selected role.

User Interfaces	Figure 5.5: Login Interface

Use case number: UC-03	**User case name:** View Profile
Brief description	The user views their personal details and their medical overview.
Actors	Patient, Doctor
Preconditions	User is logged in, and the portal is open.
Postconditions	Patient: Profile information is displayed in the Profile tab. Doctor: Practice details are displayed in the Appointments tab.
Primary path	1. User navigates to the Profile/Appointments tab.
Alternatives	None.
Exceptions	None.
Data Requirements	Same as Registration Window.
User Interfaces	Figure 5.4: Profile and Edit Interface

Use case number: UC-04	**User case name:** Edit Profile
Brief description	The user updates their editable personal information through the modal form.
Actors	Patient, Doctor
Preconditions	User is logged in and viewing their Profile/Appointments tab.
Postconditions	Updated values are saved to the database, and the respective tab refreshes with the new data.
Primary path	1. User clicks ‘Edit Profile’. 2. User edits fields 3. Click ‘Save’.
Alternatives	User clicks ‘Cancel’; no changes saved.
Exceptions	Field left empty.
Data Requirements	Patient: contact number, home address, emergency contact details, medical overview. Doctor: first/middle/last name, contact number.
User Interfaces	Figure 5.4: Profile and Edit Interface

Appointment Subsystem

Use case number: UC-05	**User case name:** View Appointments
Brief description	The user views their list of past and upcoming appointments with their status.
Actors	Patient, Doctor
Preconditions	User is logged in.
Postconditions	The appointment list is shown in the Appointments tab.
Primary path	1. User opens Appointments tab. 2. User can sort by date (ASC/DESC).
Alternatives	None.
Exceptions	None.
Data Requirements	AppointmentID, date, time, doctor/patient name, location, reason, status.
User Interfaces	Figure 5.7: Appointments and Reschedule Interface

Use case number: UC-06	**User case name:** Book Appointment
Brief description	A patient selects a doctor, date, and available time slot to create a new appointment.
Actors	Patient
Preconditions	Patient is logged in. At least one Doctor account exists in the database.
Postconditions	A new Appointments row with Pending status and a linked Invoices row are inserted in a single transaction.
Primary path	1. Patient opens the Appointments tab -> Book New Appointment. 2. Select doctor and date. 3. Patient picks an open slot and enters the consultation reason. 4. Clicks ‘Confirm Appointment’.

Alternatives	No time slot appears as available in the dropdown if all slots are booked for the chosen date.
Exceptions	Empty fields.
Data Requirements	DoctorID, appointment date, time slot, location, reason, and consultation fee.
User Interfaces	Figure 5.7: Appointments and Reschedule Interface

Use case number: UC-07	**User case name:** Reschedule / Cancel Appointment
Brief description	The user reschedules an existing appointment to a new date and time or cancels an appointment.
Actors	Patient, Doctor
Preconditions	The appointment exists and has a non-terminal status (Pending or Rescheduled). Cancelled and completed appointments cannot be updated.
Postconditions	Appointment status is updated to Rescheduled or Cancelled. Cancelled appointments trigger automatic removal of linked invoices.
Primary path	Reschedule: 1. User clicks 'Reschedule' on an eligible appointment. 2. User selects new date/time. 3. User confirms. Appointment view refreshes. Cancel: 1. User selects a row in the Appointment table. 2. User clicks 'Cancel.' 3. User confirms. Appointment view refreshes.
Alternatives	User clicks 'Cancel' or 'No' in the reschedule or cancellation modal; no changes are made.
Exceptions	Selected slot is unavailable.
Data Requirements	AppointmentID, new date, new time slot, updated status.
User Interfaces	Figure 5.7: Appointments and Reschedule Interface

Clinical Subsystem

Use case number: UC-08	**User case name:** Start Consultation / Prescribe Medications & Write Diagnosis
Brief description	The user starts a consultation from a specific Pending appointment, which pre-loads patient context into the Prescriptions tab for medication, diagnosis, and laboratory request entry.
Actors	Doctor
Preconditions	The doctor is logged in. A scheduled appointment exists.
Postconditions	MedicalRecords, Prescriptions, and LabRequests rows are inserted. Appointment status is set to Completed. Invoice status updates to Processing. The doctor's digital signature is saved as a PNG file.
Primary path	1. Doctor clicks 'Start Consult'. 2. The doctor enters diagnosis, medication, lab requests, and notes. 3. Doctor signs on canvas. 4. Clicks 'Save & Add to Record.'
Alternatives	None.
Exceptions	Empty diagnosis field. Empty signature canvas.
Data Requirements	AppointmentID, PatientID, diagnosis text, notes, drug name, dosage, instructions, lab request descriptions, signature PNG.
User Interfaces	Figure 5.9: Prescriptions Interface

Use case number: UC-09	**User case name:** View Medical History
Brief description	A patient views their full consultation history (e.g., diagnosis, prescriptions, and laboratory requests) by selecting a visit
Actors	Patient
Preconditions	Patient is logged—at least one completed appointment.

Postconditions	The medical history list is displayed.
Primary path	1. Patient opens Medical Records tab. 2. Patient clicks on a past visit.
Alternatives	None.
Exceptions	None.
Data Requirements	AppointmentID, diagnosis, prescriptions, lab requests, doctor's name, and consultation details.
User Interfaces	Figure 5.8: Medical Records Interface

Use case number: UC-10	**User case name:** View Patient Records
Brief description	A doctor searches and clicks on a patient row to view their full demographic and medical overview summary.
Actors	Doctor
Preconditions	The doctor is logged in. A patient with at least one completed appointment with this doctor.
Postconditions	The patient summary is displayed.
Primary path	1. Doctor opens the Patient Records tab. 2. Doctor selects a patient.
Alternatives	The doctor uses the search box to filter by patient name.
Exceptions	None.
Data Requirements	PatientID, demographics, blood type, emergency contact, and initial medical overview.
User Interfaces	Figure 5.8: Medical Records Interface

Use case number: UC-11	**User case name:** Download Rx & Lab PDF
Brief description	A user downloads a PDF document containing the medical details of a completed appointment.
Actors	Patient, Doctor
Preconditions	The appointment is marked as completed.
Postconditions	A PDF file is generated and opened with the OS default PDF viewer.
Primary path	1. User selects a completed visit from the Medical/Patient Records tab. 2. Clicks 'Download RX & Lab'. 3. PDF saved locally and auto-opened.
Alternatives	None.
Exceptions	The signature PNG file is missing.
Data Requirements	AppointmentID, doctor name/header, patient name/age/sex, diagnosis, prescriptions, lab requests, and doctor signature.
User Interfaces	Figure 5.8: Medical Records Interface

Use case number: UC-12	**User case name:** Update Lab Request
Brief description	A doctor marks a laboratory request as Completed after results have been hypothetically reviewed.
Actors	Doctor
Preconditions	The doctor is logged in. A pending laboratory request exists.
Postconditions	The laboratory request status is updated to Completed on both portals.
Primary path	1. Doctor opens the Patient Records tab. 2. Click a patient row. 3. Locates lab request for appointment. 4. Click the status toggle, mark as Completed.
Alternatives	None.
Exceptions	None.
Data Requirements	RequestID, updated status (Completed).
User Interfaces	Figure 5.8: Medical Records Interface

Billing Subsystem

Use case number: UC-13	**User case name:** View Balances / Paid Billing
Brief description	Users view invoice history. Patients see their respective invoices; doctors see all unresolved and paid billings across their patients.
Actors	Patient, Doctor
Preconditions	User is logged in. At least one invoice exists.
Postconditions	The invoice list is displayed with statuses and the total outstanding balance.
Primary path	1. User opens Billing tab.
Alternatives	None.
Exceptions	None.
Data Requirements	InvoiceID, AppointmentID, amount, status, and date issued.
User Interfaces	Figure 5.10: Billing Management Interface

Use case number: UC-14	**User case name:** Verify Payment
Brief description	A doctor/clinic assistant (hypothetically) confirms a patient's payment, which transitions the invoice status from Processing to Paid.
Actors	Doctor
Preconditions	An invoice with a Processing status exists.
Postconditions	Invoice status updates to Paid. The patient's billing tab updates simultaneously.
Primary path	1. Doctor opens the Billing tab, Unresolved Balances. 2. Double-clicks a Processing invoice. 3. Doctor clicks 'Yes'.
Alternatives	Doctor clicks No (reject) on verifying payment, which reverts to Pending status.
Exceptions	None.
Data Requirements	InvoiceID, updated status, timestamp.
User Interfaces	Figure 5.10: Billing Management Interface

Use case number: UC-15	**User case name:** Download Invoice PDF
Brief description	The user downloads a paid invoice PDF for a fully settled appointment.
Actors	Patient, Doctor.
Preconditions	The invoice status is Paid.
Postconditions	A watermarked PDF Invoice is created and opened with the OS default PDF viewer.
Primary path	1. User opens Billing tab. 2. Selects a paid invoice. 3. Clicks 'Download Invoice PDF'. 4. PDF saved locally and auto-opened.
Alternatives	None.
Exceptions	The invoice status is not paid.
Data Requirements	InvoiceID, invoice date, amount, patient name, doctor name, consultation date, and reason.
User Interfaces	Figure 5.10: Billing Management Interface

Use case number: UC-16	**User case name:** Manage Consultation Fee
Brief description	A doctor updates their consultation fee, wherein the updated fee applies to future bookings only.
Actors	Doctor
Preconditions	The doctor is logged in and has an existing Fee row.
Postconditions	A new Fee row is inserted with the new amount and the current timestamp. Existing invoices remain unaffected.
Primary path	1. Doctor opens the Billing tab. 2. Clicks Edit Fee.

	3. Doctor types the new amount and confirms.
Alternatives	None.
Exceptions	Empty or non-numeric fee entered.
Data Requirements	DoctorID, new BaseAmount, and EffectiveDate.
User Interfaces	Figure 5.10: Billing Management Interface

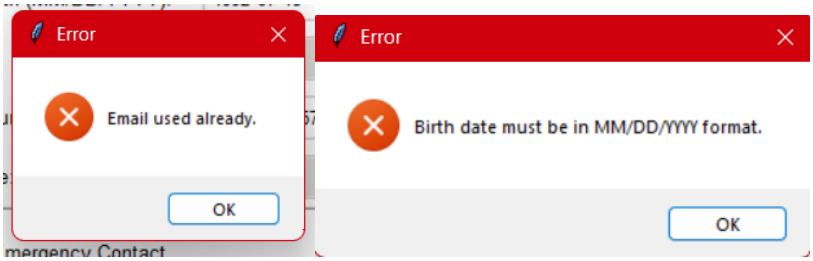
Appendix B: Data Dictionary

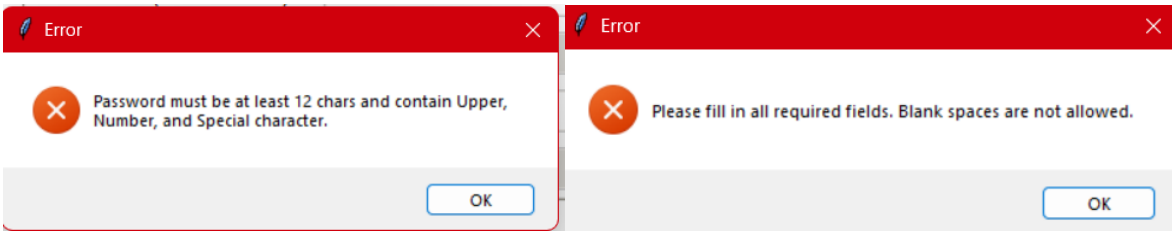
Table: Users		
Column Name	**Data Type**	**Constraints & Defaults**
UserID	int	Primary Key (Identity)
Email	nvarchar(100)	Not Null, Unique
PasswordHash	nvarchar(255)	Not Null
Role	nvarchar(20)	Not Null, CHECK ('Doctor', 'Patient')
DateRegistered	datetime2(7)	Not Null, DEFAULT getdate()
Table: Doctors		
DoctorID	int	Primary Key (Identity)
UID	int	Foreign Key (Users), Unique, Not Null
FirstName	nvarchar(50)	Not Null
MiddleName	nvarchar(50)	Not Null
LastName	nvarchar(50)	Not Null
Gender	char(1)	Not Null, CHECK ('F', 'M')
ContactNumber	varchar(11)	Not Null, CHECK (Format:'09…')
Specialization	nvarchar(100)	Not Null
LicenseNumber	nvarchar(20)	Unique, Not Null
DefaultLocation	nvarchar(100)	Not Null, DEFAULT 'General Clinic'
Table: Patients		
PatientID	int	Primary Key (Identity)
UID	int	Foreign Key (Users), Unique, Not Null
FirstName	nvarchar(50)	Not Null
MiddleName	nvarchar(50)	Not Null
LastName	nvarchar(50)	Not Null
BirthDate	date	Not Null
Gender	char(1)	Not Null, CHECK ('F', 'M')
ContactNumber	varchar(11)	Not Null, CHECK (Format:'09…')
BloodType	nvarchar(3)	Not Null
HomeAddress	nvarchar(255)	Not Null
IntialConditions	nvarchar(max)	Null
IntialAllergies	nvarchar(max)	Null
IntialMedications	nvarchar(max)	Null
Table: EmergencyContacts		
ContactID	int	Primary Key (Identity)
PID	int	Foreign Key (Patients), Unique, Not Null
FullName	nvarchar(150)	Not Null
ContactNumber	varchar(11)	Not Null, CHECK (Format:'09…')
Relationship	nvarchar(50)	Not Null
Table: Appointments		
AppointmentID	int	Primary Key (Identity)
PID	int	Foreign Key (Patients), Not Null
DID	int	Foreign Key (Doctors), Not Null
AppointmentDateTime	datetime2(7)	Not Null
Location	nvarchar(100)	Not Null
Reason	nvarchar(255)	Not Null
Status	nvarchar(20)	Not Null, CHECK ('Pending, 'Completed', 'Cancelled', 'Rescheduled')
Table: MedicalRecords		
RecordID	int	Primary Key (Identity)
AID	int	Foreign Key (Appointments), Unique, Not Null
Diagnosis	nvarchar(500)	Null
Notes	nvarchar(max)	Null

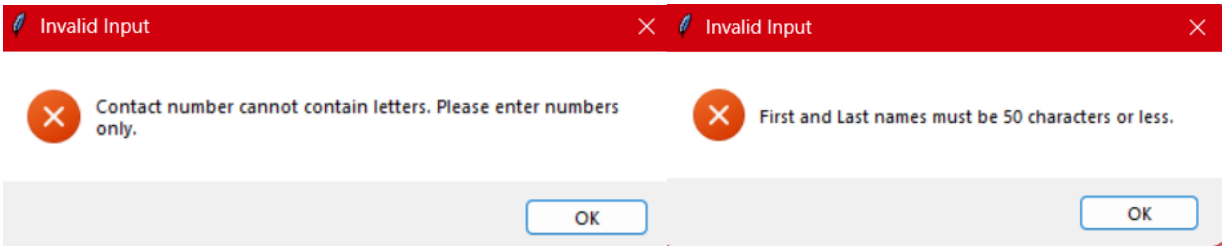
DateRecorded	datetime2(7)	Not Null, DEFAULT getdate()
Table: Prescriptions		
PrescriptionID	int	Primary Key (Identity)
AID	int	Foreign Key (Appointments), Not Null
DrugName	nvarchar(100)	Null
Dosage	nvarchar(50)	Null
Instructions	nvarchar(255)	Null
DatePrescribed	datetime2(7)	Null, DEFAULT getdate()
Table: LabRequests		
RequestID	int	Primary Key (Identity)
AID	int	Foreign Key (Appointments), Not Null
Description	nvarchar(max)	Null
DateRequested	datetime2(7)	Null, DEFAULT getdate()
Status	nvarchar(20)	Null, DEFAULT 'Pending', CHECK ('Pending', 'Completed')
Table: Fee		
FeeID	int	Primary Key (Identity)
DID	int	Foreign Key (Doctors), Not Null
BaseAmount	decimal(10,2)	Not Null
EffectiveDate	datetime2(7)	Null, DEFAULT getdate()
Table: Invoices		
InvoiceID	int	Primary Key (Identity)
AID	int	Foreign Key (Appointments), Unique, Not Null
Amount	decimal(10,2)	Null
Status	nvarchar(20)	Not Null, CHECK ('Pending, 'Processing, 'Paid', 'Cancelled')
DateIssued	datetime2(7)	Null, DEFAULT getdate()

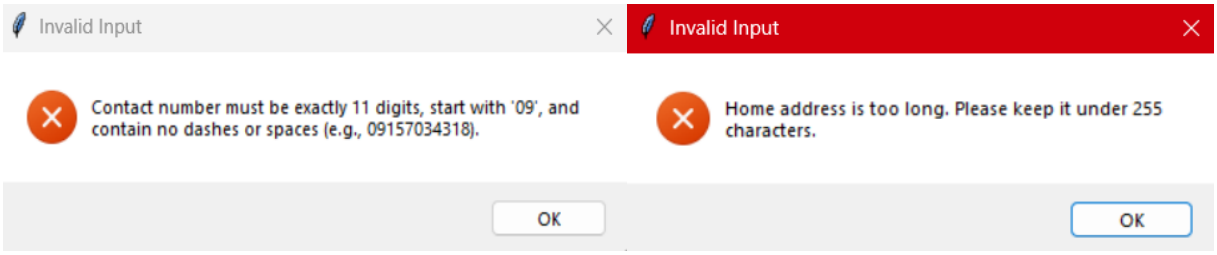
Appendix C: Data Validation and Exception Handling Evidence

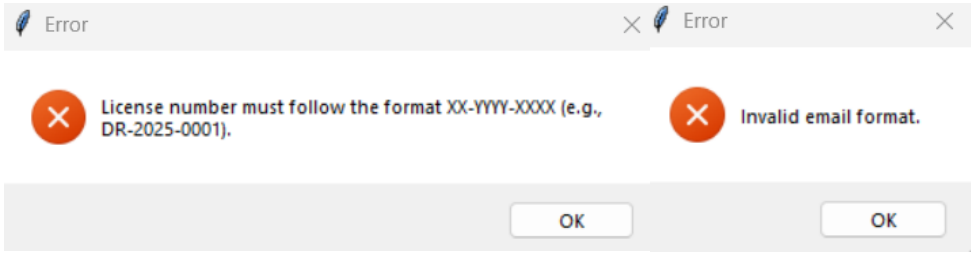
Registration Forms and Profile Editor (Patient/Doctor Portal)

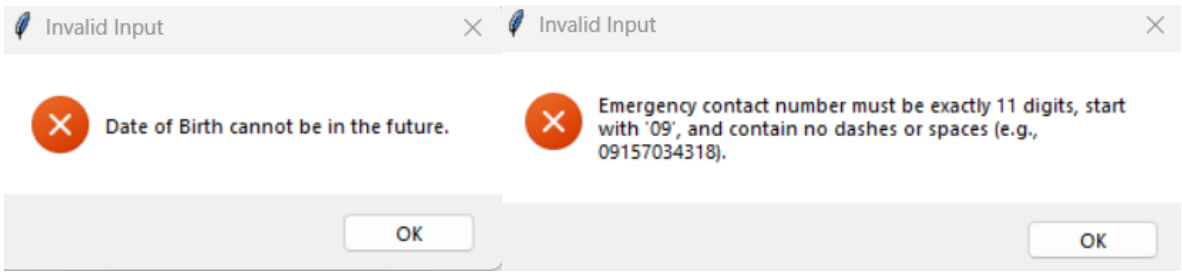


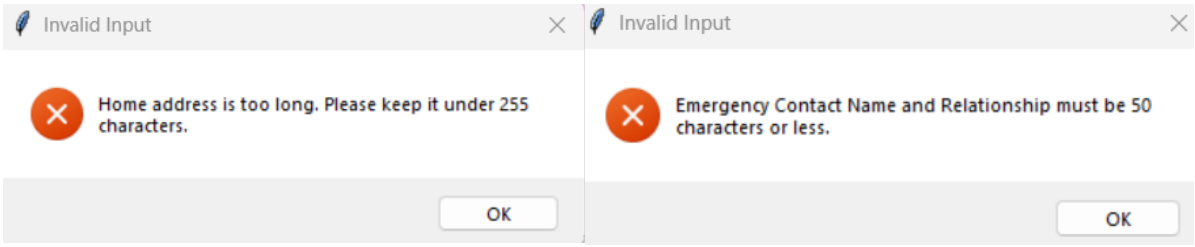


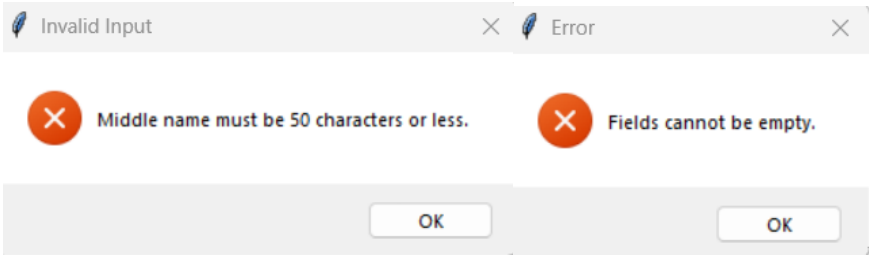




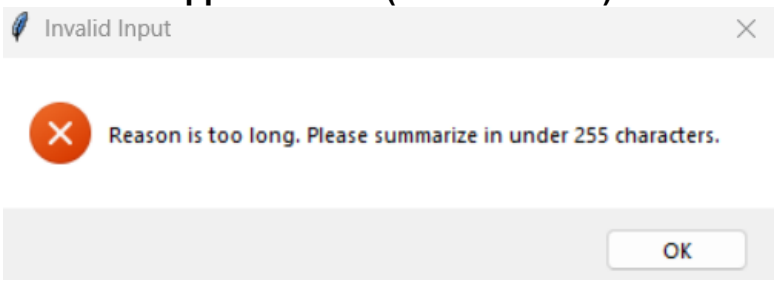




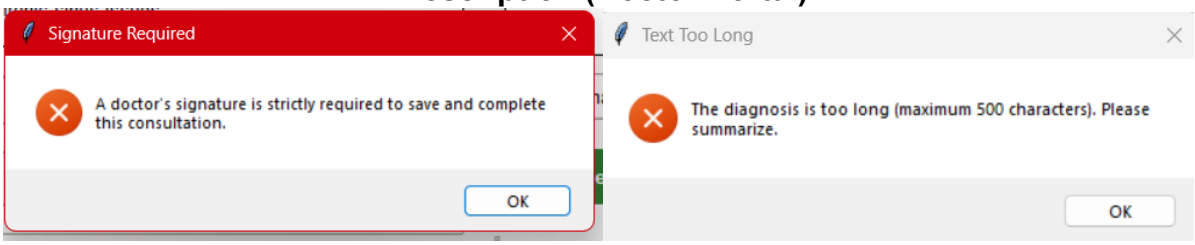


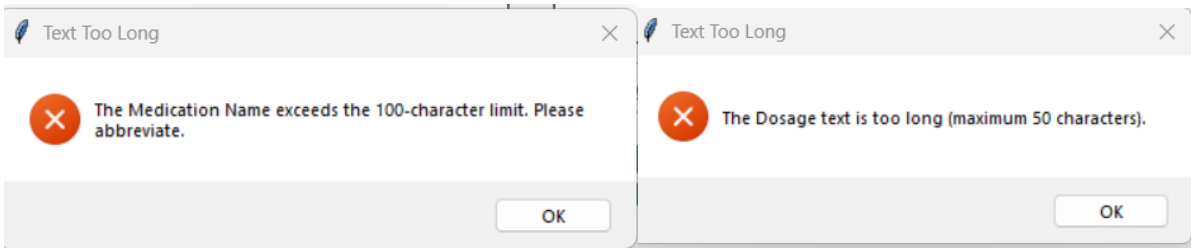


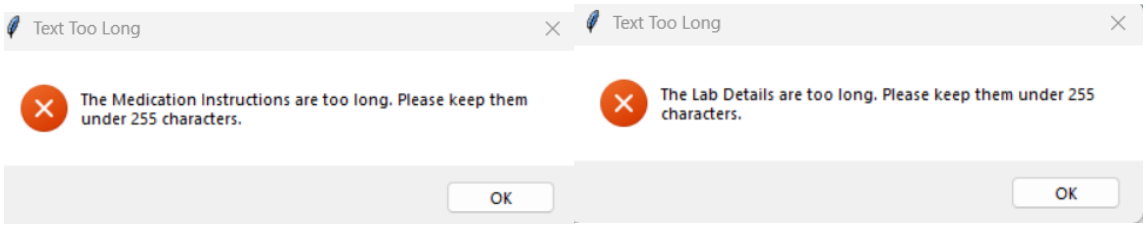
Appointments (Patient Portal)



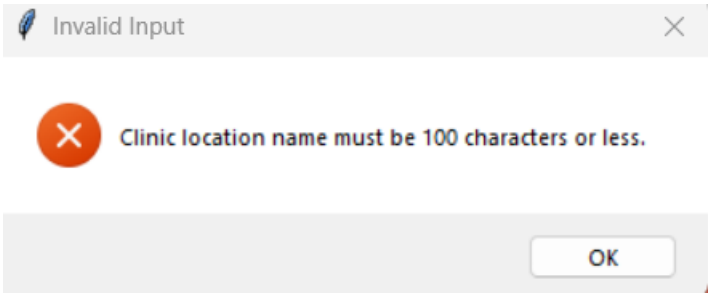
Prescription (Doctor Portal)



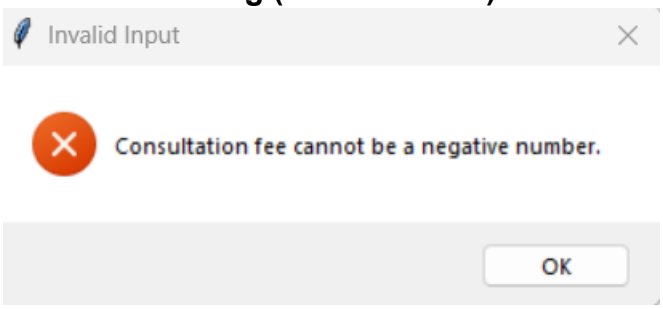




Doctor's Location



Billing (Doctor Portal)



Appendix D: AI Usage Documentation

my python app successfully runs inside pycharm, but when it is ran through the regular windows command prompt (python main.py), it says ModuleNotFoundError: No module named 'pyodbc'. why cant the command prompt see the packages installed in pycharm?

It can be incredibly frustrating to have your application working flawlessly in your IDE, only to throw a `ModuleNotFoundError` the second you try to run it from the command line. It feels like the terminal is just ignoring all the work you've already done.

When you're building an offline desktop application that needs to talk to a local SQL Server, `pyodbc` is essential for bridging that gap—so getting the terminal to recognize it is a critical step.

Here is exactly why this is happening and how to fix it.

The Culprit: Virtual Environments (venvs)

The reason the command prompt can't see the packages installed in PyCharm is because PyCharm is highly organized and uses **Virtual Environments**.

When you create a project in PyCharm, it automatically creates an isolated "bubble" (a virtual environment) for that specific project. When you installed `pyodbc` in PyCharm, it was installed *only* inside that bubble, not on your computer's main, system-wide Python installation.

When you open a regular Windows Command Prompt and type `python main.py`, Windows uses the system-wide Python installation, which currently has no idea what `pyodbc` is because it's looking outside the bubble.

How to Fix It

To make your command prompt see `pyodbc`, you need to tell it to step inside PyCharm's virtual environment bubble before you run your script.

Step 1: Open the Command Prompt and navigate to your project
Open your regular Windows command prompt (`cmd`) and use the `cd` command to navigate to the folder where your `main.py` is located.

```
cd path\to\your\project\folder
```

in my app, i open a toplevel window for Patient Registration. but when i submit and the window closes, the main window does not show the patient data. what are your suggestions in managing the state and passing data back from the toplevel window to the gui class?

Managing state between multiple windows is a classic hurdle in desktop application development. The issue you are running into happens because the main window doesn't automatically know when the `Toplevel` window has finished its work or what data it processed.

To maintain clean Object-Oriented Programming (OOP) principles and preserve the separation of concerns in a Three-Tier Architecture, you want to avoid tightly coupling the main GUI class to the Toplevel class.

Here are three highly effective strategies to manage this state, depending on how your Presentation tier interacts with your Data Access Object (DAO) layer.

1. The Database-Driven Refresh (`wait_window`)

If your application relies on a local SQL Server as the single source of truth, the cleanest approach is to let the database handle the state. The main window pauses, waits for the Toplevel to close, and then asks the DAO for the updated list.

How it works:

1. The main window opens the `PatientRegistration` Toplevel.
2. The main window calls `.wait_window()` on the Toplevel instance, which pauses the main window's execution flow at that exact line.
3. The Toplevel collects the data, passes it to the Business Logic layer, saves it via the DAO, and then destroys itself.
4. The main window resumes execution and immediately calls a method to refresh its data grid from the database.

my username and password labels in the login form window are misalignied with their entry boxes due to the text length differences. how do i use the sticky parameter and padx/pady methods better to align the widgets.

Tkinter's `grid` geometry manager is incredibly powerful, but it can definitely feel stubborn when you're trying to make a clean, professional-looking form.

The misalignment happens because, by default, the `grid` manager places widgets in the absolute center of their allocated "cell." Since the word "username" and "password" have different character lengths, centering them makes their right edges—where they meet the entry boxes—completely jagged.

To fix this, you need to tell the labels to "stick" to a specific side of their cell, and then use padding to give them some breathing room.

1. The `sticky` Parameter (Anchoring)

The `sticky` parameter uses compass directions (`"n"` , `"s"` , `"e"` , `"w"`) to dictate where a widget sits inside its cell.

- To make your labels align cleanly against the entry boxes, you want them to right-align. In compass terms, this is East.
- You should use `sticky="e"` for your labels.
- Alternatively, if you want left-aligned labels, you would use `sticky="w"` (West).

2. The `padx` and `pady` Parameters (Spacing)

Without padding, widgets will squash right up against the edges of the window or each other.

- `padx` adds horizontal spacing (left and right of the widget).
- `pady` adds vertical spacing (top and bottom of the widget).
- You can pass a single integer (e.g., `padx=10`), which applies 10 pixels to *both* sides, or a tuple (e.g., `padx=(10, 5)`), which applies 10 pixels to the left and 5 to the right.

Appendix E: Consultation Timeline with Research Advisor

11 Weeks Consultation Time										
Week1	Week 2	Week3	Week4	Week5	Week6	Week7				Week 8
Intial Meeting and Overall Briefing	Checking of Part 1 Paper & Make Dev't Timeline	Check Edit Profile Patient & Check Edit Profile Doctor	Holy Week	Holiday	Edit Profile Doctor - ok Check Appointments Function Patient - ok Check Appointments Function Doctor - ok	Check Medical History Patient	Check Medical History Doctor	Billing Functionality Patient	Billing Functionality Doctor	Check Errors & Check Parts of Paper (Chapters 1 to 3) & Outline Workplan for Paper
Friday, 13 March 2026	Thursday, 19 March 2026	Thursday, 26 March 2026	Thursday, 2 April 2026	Thursday, 9 April 2026	Thursday, 16 April 2026	Tuesday, 28 April 2026				Thursday, 7 May 2026
Done	Done	Done	Done	Done	Done	Done	Done	Done	Done	Done
								Pop Up Screen Saying Confirm Payment		Validation Entry Error (Doctor POV)
										Patient - Reshedule Button should not work on a Cancelled Date & Time by the Doctor.
What to check:										Remove Pay Now
Patient Module										
Front End Interface										
Program Algorithm										
Back End Database										
Doctor's Module										
Front End Interface										
Program Algorithm										
Back End Database										

Week 9	Week 10	Week11	
Chapter 4 & 5	Chapter 6 (Testing) & Chapter 7 (Conclusion & Future Work)	Abstract & Appendices	Submission of Paper & System
Thursday, 14 May 2026	Friday, 22 May 2026	Thursday, 28 May 2026	Monday, 1 June 2026
Done	Done		
	4:00 pm	11:00 am	